KAPASA MAKASA UNIVERSITY
SCHOOL OF APPLIED SCIENCES AND OPEN LEARNING
DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY (ICT)

CYS 425

MOBILE DEVICE SECURITY AND FORENSICS

CONSOLIDATED EXAM STUDY GUIDE

Critical Theory + Practical Implementation, Organised for Scenario & Application-Based Assessment

Prepared by: Darry D.

How to Use This Guide

This guide compresses all three source materials into one revision document, organised by topic unit rather than by source, because your exam is scenario- and application-based — you will be asked to reason through a situation, not recite a chapter title. Each unit follows the same structure: a theory Overview, Key Concepts (the compressed 'must-know' bullets), a Critical Terms table, a Practical Angle (commands/code where relevant), a worked Scenario Question with a full Model Answer, and an Exam Tip flagging how the topic tends to be tested.

Read every unit once for understanding, then use the Scenario Bank (Part XXXIII) and Essay Bank (Part XXXIV) as timed practice, and the Rapid Revision Checklist (Part XXXVI) as your final-day, no-new-reading pass.

How to Get the Most Out of Each Unit

- Don't skip straight to the Scenario Question. The Overview and Key Concepts exist because scenario answers are graded on whether you can correctly NAME the underlying concept, not just reason around it vaguely — read the theory first.
- After reading a unit's Model Answer, cover it and try to reconstruct the answer's SHAPE (see Part XXX) from the question alone before re-reading it. Recognising a question type matters as much as recalling the fact.
- Use the Critical Terms tables as flashcards on Day 8 (see the study plan below) — cover the definition column and test yourself against the term alone.
- Cross-reference the Common Exam Traps in Part XX regularly; several units explicitly link back to a trap, and recognising a trap in a live exam question is often worth more than any single memorised fact.

A Note on the Two Tracks

Parts I–V, XIV–XV, XVIII, XXI–XXII, and XXVIII belong to the Mobile Forensics track (drawn from Mobile Forensic Investigations). Parts VI–XIII, XVI–XVII, XXIII–XXVII belong to the Hacking/Reverse-Engineering track (drawn from the Android Application Reverse Engineering & Obfuscation lab document and Hacking Android). Parts XIX, XXIX, and XXX deliberately combine both tracks, because your exam is very likely to do the same in at least one question — do not study the two tracks in complete isolation from one another.

Suggested 10-Day Study Plan

Day	Focus	Tasks
Day 1	Forensics Foundations	Read Parts I–II (foundations, seizure, toolbox forensics, acquisition tiers). Memorise the 4-stage process and the 3 acquisition tiers cold.
Day 2	SIM/Device Classes + SQLite	Read Parts III–IV. Drill the ICCID vs IMSI distinction and the WAL/journal concept until you can explain both without notes.
Day 3	Advanced Analysis + Presentation	Read Part V. Practise writing a 3-sentence executive summary for a fictional case in your own words.
Day 4	Lab Setup + App Architecture + Rooting	Read Parts VI–VII. Rebuild the PATH configuration and the rooting workflow from memory on paper.
Day 5	Full RE Workflow (the big one)	Read Part VIII twice. Re-type the entire build→disassemble→modify→repackage command sequence from memory, then check it against the guide.
Day 6	Obfuscation + Dynamic Analysis	Read Parts IX–X. Focus on WHY packing defeats static analysis and WHY dynamic analysis is the answer.

Day	Focus	Tasks
Day 7	Component Attacks + Malware/Device Attacks	Read Parts XI–XII. Drill the exported-component pattern and the malware-indicator pattern (permission mismatch + DexClassLoader).
Day 8	Command Cheat-Sheet + Glossary	Go through the full command cheat-sheet and glossary; flag any term you cannot define in one sentence and re-read its source unit.
Day 9	Scenario Bank — timed practice	Attempt the full scenario bank under timed conditions (≈3 minutes/question) without looking at the model answers first, then mark yourself.
Day 10	Rapid Revision + Weak-Spot Pass	Use the Rapid Revision Checklist as a final pass. Re-read only the units tied to any scenario you got wrong on Day 9.

Contents Overview

Part	Track	Title
I	Forensics	Foundations: What Makes Mobile Forensics Different
I	Forensics	Lawful Seizure and Scene Documentation
II	Forensics	Toolbox Forensics: The Multiple-Tool Approach
II	Forensics	Acquisition Tiers: Logical, File-System, and Physical
III	Forensics	SIM/UICC Analysis and Legacy Device Classes
III	Forensics	iOS, IoT, Wearables, and Drones as Evidence Sources
IV	Forensics	SQLite Forensics and Scripted Analysis
IV	Forensics	Advanced Android Forensic Analysis and Corroboration
V	Forensics	Presenting Mobile Forensic Evidence
VI	Hacking/RE	Building the Android Security Testing Lab
VI	Hacking/RE	Android Application Architecture and the Sandbox Model
VII	Hacking/RE	Android Rooting
VIII	Hacking/RE	Full APK Reverse-Engineering Workflow: Build → Disassemble → Modify → Repackage
IX	Hacking/RE	Obfuscation, Packing, and Their Limits
X	Hacking/RE	Dynamic Analysis: Proxies, Frida, Xposed, and Drozer
XI	Hacking/RE	Attacking App Components and Server-Side Interfaces
XII	Hacking/RE	Android Malware Analysis and Device-Level Attacks
XIII	Hacking/RE	The OWASP Mobile Top 10 (2014) in Full
XIV	Forensics	Legal, Ethical, and Professional Standards Deep-Dive
XV	Forensics	Acquisition Tiers and the Commercial Tool Landscape, Compared
XVI	Hacking/RE	Deep-Dive: Exploiting a Vulnerable Content Provider
XVII	Hacking/RE	AndroidManifest.xml and the Permission Model
XVIII	Forensics	iOS vs Android: Forensic Acquisition Compared
XIX	Both / Capstone	Capstone Case Study: Investigating a Repackaged, Trojanised Application
XX	Both	Common Exam Traps and Distractors
XXI	Forensics	Feature Phones, BlackBerry, and Windows Phone: Legacy Device Forensics
XXII	Forensics	Cloud Forensics and Multi-Device Evidence Correlation
XXIII	Hacking/RE	Android Backup Internals: .ab Format and the Android Backup Extractor
XXIV	Hacking/RE	Malware Indicator Patterns and Signature-Style Detection Logic
XXV	Hacking/RE	Server-Side Security for Mobile App Backends
XXVI	Hacking/RE	Drozer: The Standard Assessment Workflow, End to End
XXVII	Hacking/RE	Encryption and Key Management Failures on Mobile

Part	Track	Title
XXVIII	Forensics	Advanced Physical Acquisition: Chip-Off and JTAG Techniques
XXIX	Both / Applied Synthesis	When Forensics Meets a Live Security Incident
XXX	Both / Exam Skills	Exam Technique: Structuring a Scenario Answer for Maximum Marks
XXXI	Reference	Quick-Reference Comparison Tables
XXXII	Reference	Consolidated Command & API Cheat-Sheet
XXXIII	Practice	Scenario Bank (Mixed, Timed Practice)
XXXIV	Practice	Essay-Style Practice Questions (Full Scenario Simulation)
XXXV	Reference	Glossary of Critical Terms
XXXVI	Revision	Rapid Revision Checklist

PART I: FOUNDATIONS: WHAT MAKES MOBILE FORENSICS DIFFERENT

Track	Source
Forensics	Mobile Forensic Investigations, Ch. 1–2

Overview

Mobile forensics is the discipline of recovering, preserving, and interpreting digital evidence from mobile devices — smartphones, tablets, feature phones, SIM cards, and increasingly wearables and IoT devices — in a way that is legally defensible. It differs sharply from classical computer forensics. A desktop or laptop can usually be powered off, imaged bit-for-bit with a hardware write-blocker, and examined on a stable, well-documented file system. A mobile device is a moving target: it is frequently encrypted by default, constantly synchronising with cloud services, capable of being remotely wiped, and built on proprietary flash storage where a true forensic 'image' in the classical sense is often impossible to obtain without specialised chip-off or JTAG techniques.

The examiner must also contend with device diversity: thousands of Android OEM/model/firmware combinations, several iOS generations, legacy feature phones, and embedded operating systems, each with different acquisition constraints. This is why the field is built around a process (seizure → collection → analysis/examination → presentation) rather than a single acquisition technique — the correct method depends entirely on the device in front of you.

Key Concepts (Compressed)

- Mobile forensics vs. computer forensics: volatility, encryption-by-default, remote-wipe risk, and lack of a universal write-blocker are the defining differences.
- The four-stage forensic process: Seizure → Collection → Analysis/Examination → Presentation.
- Professional bodies referenced for standards and certification: IACIS (International Association of Computer Investigative Specialists) and ISFCE (International Society of Forensic Computer Examiners).
- A 'forensic' tool is not automatically forensically sound merely because it is marketed as one — validation and verification remain the examiner's responsibility.
- History matters for exam recall: mobile telephony traces back to Martin Cooper's 1973 handheld cellular call, a frequently tested trivia anchor for 'firsts' in mobile device history.

Critical Terms

Term	Definition
Volatility	The tendency of device state (RAM, active sessions, unsaved app state) to change or be lost when a device loses power or is left running.
Write-blocker	Hardware/software control that prevents any write operation from reaching original evidence media during acquisition; rarely available in true form for mobile internal flash.
IACIS / ISFCE	Professional certifying bodies for computer/mobile forensic examiners, referenced as sources of accepted standards and codes of ethics.
Chain of custody	The unbroken, documented trail of who possessed evidence, when, and why, from seizure to court presentation.

Worked Scenario Question

1. **Q1.** *You are handed a locked Android phone that is still powered on and connected to Wi-Fi. Explain, in the correct order, the first three actions you should take and why each matters.*

Model Answer: (1) Isolate network connectivity immediately — place the device in Airplane Mode or a Faraday bag/enclosure to prevent remote wipe or evidence alteration via push notifications/MDM commands; do this before anything else because every second of live network access is a risk window. (2) Photograph/document the device's current screen state, visible notifications, and battery level before touching settings, since this state itself is evidentiary and may change once you interact with the device. (3) Maintain power — connect to a portable power source if extraction will not begin immediately, since losing power on an encrypted device before a logical/AFU extraction can move it into a much harder Before-First-Unlock (BFU) state.

Note: *When a scenario opens with "a device is seized while powered on," the examiner almost always wants you to name network isolation before anything else — do not lead with acquisition method.*

PART I: LAWFUL SEIZURE AND SCENE DOCUMENTATION

Track	Source
Forensics	Mobile Forensic Investigations, Ch. 5–6

Overview

Seizure is where most defensible cases are won or lost, because a technically perfect extraction is worthless if the seizure itself was unlawful or undocumented. Before touching a device, the examiner must know the legal basis for the search — consent, a warrant with a defined scope, or exigent circumstances — and must not exceed that scope. Devices, SIM cards, and removable media should each be treated and tagged as independent evidence items, since they can be examined separately and may travel through custody separately.

Scene documentation (photographs of the device in place, its connections, and its screen state) must happen before the device is moved or interacted with. Any interaction that is unavoidable (e.g. entering Airplane Mode) must itself be documented — what was done, by whom, when, and why.

Key Concepts (Compressed)

- Legal authority to search must be established and scoped before any technical action is taken.
- Faraday bags/enclosures and Airplane Mode are the two practical options for isolating a device from cellular/Wi-Fi networks; know the trade-off (Airplane Mode requires touching an unlocked device; a Faraday enclosure does not, but can drain battery faster searching for signal).
- Every accessory (charger, SIM, SD card) is a distinct evidence item requiring its own tag and chain-of-custody entry.
- Photographing screen state, notification content, and physical damage before movement is a non-negotiable step.

Critical Terms

Term	Definition
Exigent circumstances	An emergency justification for warrantless seizure/search, e.g. imminent destruction of evidence.
Faraday enclosure/bag	A signal-shielding container that blocks cellular, Wi-Fi, and Bluetooth signals from reaching a device.
Scope of warrant	The legally defined boundary of what may be searched/seized; exceeding it can render evidence inadmissible.

Worked Scenario Question

1. **Q1.** *A colleague suggests turning off a seized, unlocked Android phone immediately to 'be safe.' Explain why this could be the wrong call.*

Model Answer: Powering off an unlocked (After-First-Unlock/AFU) device can push it into a Before-First-Unlock (BFU) encryption state on reboot, where far less data is accessible without the passcode, and some acquisition methods that worked in the AFU state (logical extraction, some file-system extractions) will no longer be possible. The better-established practice is to keep the device powered and isolated from networks (Airplane Mode or Faraday enclosure) rather than shutting it down, unless a specific, documented reason (e.g. active remote-wipe attempt underway that isolation cannot stop) justifies the trade-off.

Note: *Any scenario question about 'should I power off the device?' is testing AFU vs BFU state — always weigh loss of access against the isolation already achieved by Airplane Mode/Faraday.*

PART II: TOOLBOX FORENSICS: THE MULTIPLE-TOOL APPROACH

Track	Source
Forensics	Mobile Forensic Investigations, Ch. 7–8

Overview

No single commercial forensic suite (Cellebrite UFED, Magnet AXIOM, Oxygen Forensics, XRY, etc.) supports every device, every data type, and every parsing edge case correctly. The 'toolbox' philosophy treats this as a given: examiners are expected to cross-validate significant findings using at least one independent method — a second tool, a manual hex-level check, or a raw SQL query — before relying on a parsed result in a report. Tool verification (does the tool do what its documentation claims, in general) and tool validation (does it produce a correct result for this specific case/data set) are two distinct QA steps examiners are expected to perform and document.

Key Concepts (Compressed)

- No single tool is authoritative; cross-validation across at least two independent methods is expected practice.
- Tool verification = general capability check. Tool validation = case-specific correctness check.
- Under-parsed or misparsed fields are a known risk with any commercial forensic tool, especially for less common apps or after an OS update changes a database schema.

Critical Terms

Term	Definition
Verification	Confirming a tool behaves as documented under known/controlled test conditions.
Validation	Confirming a tool's output is correct for the specific evidence/case at hand.
Toolbox approach	Deliberately using multiple independent tools/techniques rather than relying on one, to catch parsing errors.

Worked Scenario Question

1. **Q1.** *Your primary forensic tool reports zero SMS messages recovered from a suspect handset that the case file says was actively used for messaging. What should you do before concluding the phone had no SMS activity?*

Model Answer: Treat the zero result as a parsing failure hypothesis first, not a factual finding. Cross-validate with a second, independent method: run a second commercial tool against the same extraction, or manually open the messaging app's SQLite database (see Part VII) and query the relevant table directly. Also confirm the extraction type actually reached the messaging app's data directory (a logical extraction may not have pulled non-default messaging apps). Only report 'no SMS activity found' once at least one independent method corroborates the absence.

Note: *Any scenario where a tool reports an implausible zero/empty result is testing whether you reach for independent validation before accepting the tool's output.*

PART II: ACQUISITION TIERS: LOGICAL, FILE-SYSTEM, AND PHYSICAL

Track	Source
Forensics	Mobile Forensic Investigations, Ch. 9–10

Overview

Acquisition methods sit on a spectrum of depth and risk. Logical acquisition (e.g. via ADB backup/pull, or a vendor tool's logical mode) retrieves data exposed through the OS's normal APIs — contacts, call logs, accessible app data — and is the fastest, lowest-risk, but shallowest method: it will not recover deleted records or data outside the API surface. File-system acquisition retrieves the full accessible file system, including SQLite journal/WAL files and cached data not exposed via API, typically requiring elevated (root/jailbreak-adjacent) access. Physical acquisition attempts a bit-level copy of the underlying flash storage, potentially recovering deleted and unallocated data, but is the most invasive, the hardest to achieve on modern encrypted devices, and carries the highest risk of altering evidence if done incorrectly.

Preparation before any acquisition includes confirming device identifiers (make, model, OS version, IMEI/serial), ensuring USB debugging/developer options are enabled where required, and having a documented, repeatable command sequence ready so the process is defensible and repeatable by another examiner.

Key Concepts (Compressed)

- Logical < File-System < Physical, in both depth of recovery and acquisition risk/difficulty.
- Logical acquisition cannot recover deleted data; only file-system/physical tiers have a chance of doing so.
- Hashing every extracted artefact immediately after acquisition (not after analysis) is the standard integrity practice.
- Pre-acquisition documentation (device identifiers, timestamps, examiner name, tool/command versions) is part of what makes an acquisition defensible.

Critical Terms

Term	Definition
Logical acquisition	Extraction via the OS's normal APIs/interfaces; fast, safe, but recovers only currently accessible data.
File-system acquisition	Extraction of the full accessible file system, typically recovering more residual/cached artefacts than logical.
Physical acquisition	Bit-level copy of underlying storage; can recover deleted/unallocated data but is hardest to achieve and riskiest.
adb backup / adb pull	ADB commands used for logical-tier extraction of app data and accessible storage on Android.

Worked Scenario Question

1. **Q1.** *A defence-side expert argues your logical acquisition should have recovered a deleted SMS message. How do you respond?*

Model Answer: Explain that logical acquisition, by design, only retrieves data currently exposed through the operating system's live APIs; a deleted record that has been removed from the active database table (even if its bytes still exist in unallocated space or a WAL/journal file) is outside what a logical extraction can see. Recovering it would require a file-system or physical acquisition capable of reading unallocated space or residual WAL/journal content, which is a

fundamentally different and more invasive tier of acquisition — not a shortcoming of the logical method, but a defined limitation of it that should have been documented before extraction began.

Note: *"Why didn't extraction method X recover deleted data Y" is one of the most common scenario framings — always answer by naming the acquisition tier's defined scope, not by second-guessing the tool.*

PART III: SIM/UICC ANALYSIS AND LEGACY DEVICE CLASSES

Track	Source
Forensics	Mobile Forensic Investigations, Ch. 11–12

Overview

A SIM/UICC card is an independent evidentiary source, separable from the handset. Key data fields include the ICCID (a unique identifier for the physical card itself), the IMSI (identifies the subscriber on the network), LOCI/EF-LOCI (Location Information, recording the last known network cell/area), and FPLMN (Forbidden PLMN list — networks the SIM was refused service on, useful for establishing rough geographic/network history). Older SIMs may also retain SMS remnants directly on the card even after deletion from the handset's messaging app, since the card has its own limited storage and deletion flags rather than true erasure.

Legacy device classes (feature phones, BlackBerry, Windows Phone) require different toolchains and often expose far less data than modern smartphones, but remain relevant in jurisdictions and case types where older hardware persists. The 'Iceberg Theory' — introduced around this point in the source material — captures the idea that the data visible through a standard extraction is only a small fraction of what is potentially recoverable; deeper analysis (file carving, unallocated space review, manual parsing) reveals the much larger mass of data 'under the waterline.'

Key Concepts (Compressed)

- ICCID identifies the card; IMSI identifies the subscriber — do not confuse the two on an exam.
- LOCI/EF-LOCI = last known network location data; FPLMN = forbidden/refused networks list.
- SMS remnants can survive on a SIM after deletion from the phone's app due to how SIM storage flags 'deleted' records.
- The Iceberg Theory: default/surface extraction reveals only a fraction of recoverable data; most is hidden until deeper analysis is applied.

Critical Terms

Term	Definition
ICCID	Integrated Circuit Card Identifier — uniquely identifies the physical SIM/UICC card.
IMSI	International Mobile Subscriber Identity — identifies the subscriber/account on the mobile network.
LOCI / EF-LOCI	Location Information elementary file on a SIM, recording the last registered network area.
FPLMN	Forbidden Public Land Mobile Network list — networks that refused the SIM service.
Iceberg Theory	The principle that surface-level extraction reveals only a small fraction of the data potentially recoverable from a device.

Worked Scenario Question

1. **Q1.** *You recover an ICCID and an IMSI from a suspect's SIM card. A junior colleague says these both identify 'the suspect's phone number.' Correct the error and explain the forensic implication of the distinction.*

Model Answer: Neither value is the phone number, and they identify two different things. The ICCID identifies the physical SIM card itself (useful for proving a specific card was used in a specific handset at a specific time). The IMSI identifies the subscriber account with the network operator (useful for correlating with call detail records held by the carrier). Confusing them matters forensically: if a suspect swaps SIM cards between handsets, the ICCID changes with the card while investigators must go to the carrier, using the IMSI, to obtain call/data records tied to the subscriber regardless of which physical card or handset was in use at a given time.

Note: *Any question pairing ICCID and IMSI is testing whether you can correctly separate 'card identity' from 'subscriber identity.'*

PART III: IOS, IOT, WEARABLES, AND DRONES AS EVIDENCE SOURCES

Track	Source
Forensics	Mobile Forensic Investigations, Ch. 13, 16

Overview

iOS devices bring their own acquisition constraints — strong default encryption, a tightly controlled file system, and acquisition options that vary sharply between an unlocked (AFU) and locked (BFU) device state. As with Android, the achievable acquisition tier depends heavily on whether the device is unlocked and whether a trusted pairing relationship (or backup password) already exists.

IoT devices, wearables (smartwatches, fitness trackers), and drones extend the evidentiary surface beyond the traditional handset: a fitness tracker's step/heart-rate/GPS logs, a smart-home hub's event log, or a drone's flight-log and telemetry data can all corroborate or contradict a timeline built from a phone. These devices frequently sync to a companion phone app and/or cloud account, so evidence may exist redundantly in three places — the device itself, the companion app's local cache on the phone, and the cloud account — each with a different acquisition method.

Key Concepts (Compressed)

- AFU (After First Unlock) vs BFU (Before First Unlock) is the critical state distinction for iOS acquisition options.
- IoT/wearable/drone evidence often exists redundantly across the device, its companion phone app, and a cloud account — consider all three.
- Companion-app local caches on a paired phone can sometimes yield data even when the wearable/IoT device itself is inaccessible.

Critical Terms

Term	Definition
AFU	After First Unlock — device state after being unlocked at least once since boot; broader data access is typically possible.
BFU	Before First Unlock — device state prior to first unlock since boot/reboot; most restrictive encryption state.

Worked Scenario Question

1. **Q1.** *A suspect's smartwatch is seized but is paired to a phone that was never recovered. What are your two most promising alternative evidence sources, and why?*

Model Answer: First, the smartwatch manufacturer's cloud account (if credentials/legal process can obtain access), since most wearables sync activity, location, and notification data to a cloud service independent of the local phone pairing. Second, any other device the same account was paired to or logged into (a tablet, a replacement phone, a family member's phone with the same companion app installed), since companion apps commonly cache recent synced data locally even on a device that isn't the primary paired phone.

Note: *Wearable/IoT scenario questions are testing whether you think in terms of 'redundant evidence across device + app + cloud' rather than treating the seized device as the only source.*

PART IV: SQLITE FORENSICS AND SCRIPTED ANALYSIS

Track	Source
Forensics	Mobile Forensic Investigations, Ch. 14

Overview

The great majority of Android and iOS application data — messages, contacts, cached credentials, location history — lives in SQLite databases. Direct SQL querying lets an examiner bypass a commercial parser's assumptions entirely and inspect the data model first-hand, which is invaluable both for validation (Part II) and for recovering data an off-the-shelf parser doesn't support. Beyond the main .db/.sqlite file, a database is frequently accompanied by a Write-Ahead Log (-wal) file and/or a rollback journal (-journal) file; these can contain committed-but-not-yet-checkpointed data, or fragments of rows that no longer appear in the main file at all — both are forensically significant and are missed if only the main database file is opened.

Beyond raw SQL, scripted analysis (commonly Python) is used to automate repetitive tasks across a case: hashing every file in a directory tree, applying regular expressions to carve structured data (phone numbers, emails, coordinates) out of unstructured blobs, and batch-processing many devices' extractions with a consistent, auditable methodology rather than manual point-and-click work.

Key Concepts (Compressed)

- Main .db file + WAL (-wal) + rollback journal (-journal) together represent the true current and recent state of a SQLite database — examine all three.
- Opening a case database in read-write mode risks altering evidence; always work from a copy in read-only mode.
- Freelist/unallocated pages inside a SQLite file can retain fragments of deleted records even after a DELETE statement has run.
- Python scripting (hashing directories, regex extraction) is the standard way to scale analysis across large extractions.

Critical Terms

Term	Definition
WAL (Write-Ahead Log)	A SQLite journaling mode file holding recent writes not yet merged into the main database file; may contain data not visible in the main file.
Rollback journal	An alternate SQLite journaling file used to support transaction rollback; can retain residual data.
Freelist page	An unallocated page inside a SQLite file that may still hold remnants of previously deleted data.
Regular expression (regex)	A pattern-matching syntax used to extract structured strings (emails, phone numbers, IDs) from raw or unstructured text.

Practical Angle

```
-- List all tables in a case database (read-only mode)
sqlite> .tables
sqlite> SELECT name FROM sqlite_master WHERE type='table';
```

```
-- Reconstruct a recent-activity timeline
sqlite> SELECT * FROM messages ORDER BY timestamp DESC LIMIT 50;

# Python: hash every file in an extraction directory
import hashlib, os
for root, _, files in os.walk('case001_extraction'):
    for name in files:
        path = os.path.join(root, name)
        with open(path, 'rb') as f:
            print(hashlib.sha256(f.read()).hexdigest(), path)
```

Worked Scenario Question

1. **Q1.** *A query against a suspect app's main SQLite database shows no messages after 14 March, but the case timeline requires activity up to 20 March. What is your next investigative step?*

Model Answer: Check for an accompanying -wal or -journal file alongside the main database, since recent writes (including messages sent after 14 March) may still be sitting in the write-ahead log and not yet checkpointed into the main file — a plain query against the main .db file alone would miss them. Open and query the WAL/journal file directly (or force a checkpoint on a working copy, never the original) before concluding the account was inactive after 14 March.

Note: *Any 'the database seems to stop early / is missing recent data' scenario is testing WAL/journal awareness — this is one of the most exam-favoured SQLite forensic details.*

PART IV: ADVANCED ANDROID FORENSIC ANALYSIS AND CORROBORATION

Track	Source
Forensics	Mobile Forensic Investigations, Ch. 15

Overview

Beyond single-artefact analysis, advanced Android forensics is about correlating multiple independent artefact types — SQLite databases, shared_prefs XML files, cache directories, and package-manager metadata — into one corroborated timeline. A single artefact in isolation is a claim; the same event confirmed by two or three independent sources (e.g. an app database entry, a matching system log timestamp, and a shared-preferences flag) is a finding. Examiners must also stay alert to anti-forensic indicators: a suspiciously empty log directory, a package manager entry referencing an app whose data directory has vanished, or timestamps that are internally inconsistent can all indicate deliberate tampering rather than innocent absence of data.

Key Concepts (Compressed)

- Corroboration across independent artefact types is far more persuasive than any single artefact alone.
- shared_prefs XML files often hold small but telling state (login flags, last-seen timestamps, feature toggles).
- Package-manager metadata (`pm list packages -f`) can reveal an app was installed even if its data directory is now missing or empty — itself a potential anti-forensic indicator.
- Rooted devices (see Part XII) complicate trust in on-device timestamps, since root access allows arbitrary modification of files and metadata.

Critical Terms

Term	Definition
shared_prefs	Android's lightweight key-value XML storage mechanism used by apps for small persistent settings/state.
Anti-forensic indicator	A finding suggesting deliberate tampering or evidence destruction, e.g. an implausibly empty log or mismatched metadata.
Corroboration	Independent confirmation of a fact from more than one distinct evidence source.

Worked Scenario Question

1. **Q1.** *You find a package-manager entry for a messaging app on a suspect device, but its /data/data directory is completely empty. What are two competing explanations, and how would you distinguish them?*

Model Answer: Competing explanation 1: the app was recently installed and never used (innocent, low-risk). Competing explanation 2: the app was used, and its data directory was deliberately cleared (e.g. via 'Clear Data' in Settings, a root-level file deletion, or an anti-forensic wipe tool) to destroy evidence. To distinguish them, check the package's install timestamp against other timeline evidence (was the account plausibly active during that window elsewhere — e.g. carrier records, a second corroborating app), and check for residual artefacts elsewhere (system logs, cached thumbnails, other apps' logs referencing this one) that would only exist if the app had actually been used.

Note: *Empty-directory-but-installed scenarios are a favourite because they force you to hold two explanations open and name a concrete corroborating check, rather than jumping to 'tampering' immediately.*

PART V: PRESENTING MOBILE FORENSIC EVIDENCE

Track	Source
Forensics	Mobile Forensic Investigations, Ch. 17

Overview

A technically flawless examination has no value if it cannot be understood and trusted by its audience — which may be a court, a corporate disciplinary panel, or a client with no technical background. Reports are typically structured as an executive summary (plain-language conclusion first), methodology (what was done and with what tools/versions, in enough detail to be repeatable), findings (the evidence itself, organised by topic or timeline), and a conclusion, with full technical detail (hash values, command logs, raw query output) pushed into appendices rather than the main narrative.

Testimony and cross-examination readiness matters as much as the written report: an examiner should be able to explain, in plain language and without notes, why each conclusion follows from the evidence, and should be prepared to explain the limitations of the acquisition tier used (see Part II) if challenged.

Key Concepts (Compressed)

- Executive summary in plain language comes first; technical detail belongs in appendices, not the main narrative.
- A repeatable methodology section (tool names, versions, exact commands, hash values) is what makes a report independently verifiable.
- An examiner should be able to explain any finding's basis and limitations in plain language under cross-examination.

Critical Terms

Term	Definition
Executive summary	A short, non-technical statement of the key findings and conclusion, written for a non-expert audience.
Methodology section	The detailed, repeatable account of tools, versions, and steps used, allowing another examiner to reproduce the work.

Worked Scenario Question

1. **Q1.** *During cross-examination, opposing counsel asks why you didn't perform a physical acquisition to recover deleted messages. How should you respond?*

Model Answer: Explain, in plain non-technical language, the acquisition-tier trade-off: physical acquisition was either not achievable on this device's encryption state without destructive/uncertain techniques, or was assessed as disproportionately risky to the integrity of the original evidence, and that the tier actually used (state which) was the appropriate, defensible choice given the device state and case requirements at the time — then state plainly what that tier could and could not recover, so the limitation is transparent rather than something the examiner appears to be concealing.

Note: *Presentation-track scenarios almost always want a plain-language answer that also states the acquisition tier's known limitation honestly — do not oversell what was recovered.*

PART VI: BUILDING THE ANDROID SECURITY TESTING LAB

Track	Source
Hacking/RE	Hacking Android, Ch. 1 & the Reverse Engineering lab document

Overview

A reproducible lab is the foundation for both the reverse-engineering and forensic-acquisition tracks. The standard toolchain (as used in both the Hacking Android lab-setup chapter and the Wayne State lab document) is: JDK (for keytool/jarsigner), the Android SDK with platform-tools and build-tools (adb, zipalign), Android Studio (or Eclipse with ADT) for building sample apps, an Android Virtual Device or a dedicated test handset, and a Kali Linux analysis host for the broader toolset (Drozer, Burp Suite, dex2jar/JD-GUI, SQLite browser, Frida, and vulnerable practice apps). Getting PATH configuration right (adding platform-tools, tools, and build-tools directories) is a common early stumbling block that has nothing to do with security concepts but blocks every subsequent exercise if wrong.

Key Concepts (Compressed)

- Core toolchain: JDK → Android SDK/Studio → AVD or test handset → Kali analysis host with Drozer/Burp/Frida/SQLite browser.
- PATH must include the SDK's tools/, platform-tools/, and build-tools/ directories for adb, emulator, and zipalign to work from any shell.
- An emulator is safer and faster to reset (snapshot) for iterative testing; a physical device is sometimes required to test hardware-dependent or anti-emulator-detection behaviour.

Critical Terms

Term	Definition
AVD (Android Virtual Device)	A configurable emulated Android device used for testing without physical hardware.
ADB (Android Debug Bridge)	The primary command-line bridge between a host machine and an Android device/emulator.
platform-tools / build-tools	SDK subdirectories containing adb/fastboot and zipalign/apksigner respectively.

Practical Angle

```
$ sudo apt update && sudo apt install -y default-jdk unzip
$ java -version
$ vim ~/.bashrc
  export ANDROIDSDK="$HOME/Android/Sdk"
  export PATH=$PATH:$ANDROIDSDK/tools:$ANDROIDSDK/platform-tools:$ANDROIDSDK/build-
tools/23.0.2
$ source ~/.bashrc
$ adb devices
```

Worked Scenario Question

1. **Q1.** *After installing the SDK, running `adb` from a new terminal returns 'command not found.' What is the most likely cause and fix?*

Model Answer: The SDK's platform-tools directory (which contains the adb binary) has not been added to the shell's PATH environment variable, or the shell configuration file (e.g. ~/.bashrc) was edited but not re-sourced in the current terminal session. Fix: add `export PATH=$PATH:$ANDROIDSDK/platform-tools` (and tools/, build-tools/ as needed) to the shell rc file, then run `source ~/.bashrc` or open a new terminal, and re-test with `adb devices`.

Note: *Environment/PATH questions are 'free marks' if you remember the three directories (tools, platform-tools, build-tools) — they show up disproportionately often as short-answer items.*

PART VI: ANDROID APPLICATION ARCHITECTURE AND THE SANDBOX MODEL

Track	Source
Hacking/RE	Hacking Android, Ch. 3

Overview

Every Android app runs under its own Linux user ID (UID), giving each app its own sandboxed file-system space (typically under /data/data/<package>/) that other apps cannot read or write without explicit permission grants or root access. Apps are built from four component types: Activities (UI screens), Services (background processing), Broadcast Receivers (system/app-wide event listeners), and Content Providers (structured data-sharing interfaces between apps). Since the introduction of ART (Android Runtime, replacing the older Dalvik interpreter/JIT model), apps are ahead-of-time (and later also JIT-assisted) compiled for better performance, but the underlying bytecode format (Dalvik bytecode, disassembled as 'smali' — see Part VIII) is conceptually unchanged for reverse-engineering purposes.

Storage locations of forensic/security interest include /data/app/ (installed APKs), /data/data/<package>/ (per-app private data, the highest-value forensic target), and /system/app/ (pre-installed system apps).

Key Concepts (Compressed)

- Each app gets its own UID and sandboxed /data/data/<package>/ directory — this is the core of Android's app isolation model.
- Four component types: Activity, Service, BroadcastReceiver, ContentProvider — each is a distinct attack surface when exported.
- ART replaced Dalvik as the runtime, but Dalvik bytecode/smali remains the reverse-engineering target format.
- /data/app/ = installed APKs; /data/data/<package>/ = private per-app data; /system/app/ = pre-installed system apps.

Critical Terms

Term	Definition
UID sandboxing	Each Android app is assigned its own Linux user ID, isolating its files from other apps by default.
Activity	An Android component representing a single, focused screen with a user interface.
Service	An Android component for long-running background operations without a user interface.
Content Provider	An Android component that exposes structured data to other apps through a defined, queryable interface.
ART	Android Runtime — the modern execution environment that replaced the Dalvik interpreter/JIT.

Worked Scenario Question

1. **Q1.** *An app declares a Content Provider with `android:exported="true"` and no permission requirement. What is the security implication, and how would an analyst confirm it in practice?*

Model Answer: Any other app on the device (including a malicious one with no special permissions) can query, and potentially insert/update/delete, the data this Content Provider exposes, since export with no permission

requirement removes Android's normal inter-app isolation for that component. An analyst would confirm this by inspecting AndroidManifest.xml for the exported flag and absence of a permission attribute, and then practically test it by querying the provider's URI from adb shell (e.g. via `content query --uri content://<authority>/<path>`) from an unprivileged context to see whether data is returned without any special access.

Note: *Exported-component questions (Activity, Service, Receiver, or Provider) are a recurring exam theme across both books — always answer with 'what can an attacker do' + 'how would you verify it.'*

PART VII: ANDROID ROOTING

Track	Source
Hacking/RE	Hacking Android, Ch. 2

Overview

Rooting grants full administrative (UID 0) control over the Android OS, bypassing the sandboxing model described in Part VI. The general workflow on many devices is: determine whether the bootloader is locked or unlocked (locked bootloaders on some OEMs, e.g. certain Sony models, require an OEM-specific unlock procedure and status check before anything else is possible), unlock it if necessary, flash a custom or patched recovery image, and then flash a root-granting package from that recovery. On Samsung devices specifically, the flashing tools Odin (Windows) and Heimdall (cross-platform) are the standard means of pushing recovery/firmware images to the device in download mode. Once rooted, custom ROMs can also be flashed, replacing the OEM firmware entirely.

For both security testers and forensic examiners, root access changes the trust model: a tester gains the ability to inspect protected directories and hook runtime calls; an examiner encountering an already-rooted device must treat on-device timestamps, logs, and even installed-app state with more scepticism, since root access removes the normal OS-level guarantees against tampering.

Key Concepts (Compressed)

- Rooting workflow: check/unlock bootloader → flash custom/patched recovery → flash root package → (optionally) flash custom ROM.
- Odin and Heimdall are the standard Samsung-specific flashing tools (Windows and cross-platform respectively) used to push images in download mode.
- A locked bootloader must have its unlock status checked/changed (device- and OEM-specific) before recovery/root flashing is possible.
- Root access on a device under forensic examination reduces confidence in the integrity of on-device logs/timestamps.

Critical Terms

Term	Definition
Bootloader	Low-level firmware that loads the OS at boot; its lock state gates whether custom recovery/root can be flashed.
Custom recovery	A replacement recovery-partition image (in place of the OEM stock recovery) that allows flashing arbitrary packages, including root.
Odin / Heimdall	Samsung-specific flashing utilities (Windows / cross-platform) used to push firmware or recovery images via download mode.
Download mode	A special low-level Samsung device state used for flashing firmware images with Odin/Heimdall.

Practical Angle

```
$ adb reboot bootloader
$ fastboot oem unlock              # unlock bootloader (device/OEM-specific)
$ fastboot flash recovery custom_recovery.img
```

```
$ fastboot reboot
# From custom recovery: flash the root package (e.g. via sideload/zip install)
```

Worked Scenario Question

1. **Q1.** *You are asked to root a Samsung device for a security assessment but the tester only has a Linux workstation, not Windows. Which tool should they use, and why?*

Model Answer: Heimdall, because it is the cross-platform (Windows/Linux/macOS) counterpart to Odin and performs the same download-mode flashing role for Samsung devices; Odin is Windows-only, so it is unusable on a Linux-only workstation for this task.

Note: *Any question naming a specific OEM (especially Samsung) and asking 'which flashing tool' is testing Odin (Windows) vs Heimdall (cross-platform) recall.*

PART VIII: FULL APK REVERSE-ENGINEERING WORKFLOW: BUILD → DISASSEMBLE → MODIFY → REPACKAGE

Track	Source
Hacking/RE	Reverse Engineering & Obfuscation lab document (full worked example)

Overview

This is the single most exam-relevant practical workflow in the whole module, because it is the exact sequence you performed hands-on and the one most likely to appear as a 'put these steps in order' or 'what does command X actually do' scenario question. An APK is a ZIP archive containing compiled Dalvik bytecode (classes.dex), resources, and AndroidManifest.xml. baksmali disassembles classes.dex into a directory of human-readable smali files organised by package name; smali reverses that process, reassembling edited smali back into a new classes.dex. Once reassembled, the old signature (META-INF/) must be removed, the archive re-zipped, freshly signed with a private key (via keytool + jarsigner), and byte-aligned (zipalign) before it will reinstall and run correctly.

The credential-logging exercise demonstrates a realistic trojanising technique: locating the exact smali instructions where a username/password are held in registers immediately before the login comparison, and inserting a static call to android.util.Log.d(String,String) referencing those registers, so the app silently writes plaintext credentials to the system log on every login attempt — while otherwise functioning identically to the original app.

Key Concepts (Compressed)

- Correct order: unzip APK → baksmali (dex → smali) → edit smali → smali (smali → dex) → delete META-INF/ → re-zip → jarsigner → zipalign → adb install.
- META-INF/ must be deleted before re-zipping because it holds the OLD signing certificate, which would conflict with re-signing.
- android.util.Log.d(String,String) is a static API — called in smali via `invoke-static`, not `invoke-virtual`.
- zipalign must run AFTER signing, not before — signing can change the archive's internal byte offsets.
- A repackaged APK signed with a different key cannot be installed over the original app without first uninstalling it (Android enforces signature matching for updates to the same package name).

Critical Terms

Term	Definition
APK	Android application package — a ZIP archive containing classes.dex, resources, and AndroidManifest.xml.
classes.dex	The compiled Dalvik bytecode file inside an APK.
smali	The human-readable intermediate assembly-like representation of Dalvik bytecode, and the name of the tool that reassembles it.
baksmali	The tool that disassembles classes.dex into a directory of smali files.
zipalign	A tool that aligns uncompressed data in an APK to a byte boundary, reducing runtime memory (RAM) usage.
jarsigner / keytool	JDK-bundled tools for generating a signing key (keytool) and signing a JAR/APK with it (jarsigner).

Practical Angle

```
$ unzip -d app-debug app-debug.apk && cd app-debug
$ java -jar baksmali.jar classes.dex        # -> out/ directory of smali
# ... edit LoginActivity.smali to inject a Log.d call ...
$ rm classes.dex
$ java -jar smali.jar -o classes.dex out
$ rm -r out META-INF
$ zip -r app-debug.zip ./* && mv app-debug.zip app-debug.apk
$ keytool -genkey -keystore mykey.keystore -alias mykey -keyalg RSA -keysize 2048 -validity
10000
$ jarsigner -sigalg SHA1withRSA -digestalg SHA1 -keystore mykey.keystore app-debug.apk mykey
$ zipalign -v 4 app-debug.apk app-debug-aligned.apk
$ adb uninstall <package> && adb install app-debug-aligned.apk

# smali injection to leak credentials to Logcat
const-string v9, "DEBUGTAG"
invoke-static {v9, v5}, Landroid/util/Log;->d(Ljava/lang/String;Ljava/lang/String;)I
```

Worked Scenario Question

1. **Q1.** *A student runs zipalign BEFORE jarsigner and the APK still installs, but a security-conscious grader marks this as incorrect procedure. Why does order matter here even though it 'worked'?*

Model Answer: Signing an APK after zipalign can change the archive's internal structure/offsets (jarsigner modifies the ZIP to add signature files under META-INF/), which can invalidate the byte-alignment zipalign just established — meaning the final, actually-shipped APK may no longer be properly aligned even though it happens to install and run in a lenient test environment. The documented, correct order is sign first, then zipalign last, precisely so the alignment step operates on the final byte layout of the file that will actually be distributed.

Note: *This exact 'wrong order but it still installed' framing is a classic trap question — the exam is testing whether you know WHY the documented order exists, not just whether the app happened to run.*

PART IX: OBFUSCATION, PACKING, AND THEIR LIMITS

Track	Source
Hacking/RE	Reverse Engineering lab document + Hacking Android, Ch. 5

Overview

ProGuard, built into the Android SDK, shrinks unused code and renames classes/fields/methods to short, semantically meaningless identifiers, raising the cost of manual reverse engineering without changing behaviour. It does not, however, obscure string literals, resource names, or the overall control flow shape of the app — an analyst can often still infer a great deal from these. Commercial/open-source packers go further, encrypting or wrapping the real dex payload so that a naive unzip+baksmali workflow (Part VIII) fails outright; the true payload is decrypted only at runtime, in memory, which is why dynamic analysis (Part X) becomes the relevant countermeasure against packing, where static disassembly alone is not.

Data-storage security (Hacking Android, Ch.5) is a closely related defensive topic: shared_prefs, SQLite databases, and files on internal/external storage are all points where sensitive data (tokens, credentials, PII) can be left unencrypted at rest; `adb backup` further exposes this data at the logical-acquisition tier unless the app explicitly opts out of Android's auto-backup mechanism.

Key Concepts (Compressed)

- ProGuard renames identifiers and strips unused code but does NOT protect string literals or resource names from casual inspection.
- Packers encrypt/wrap the real payload, defeating simple static disassembly; the payload is only revealed in memory at runtime.
- Dynamic analysis (not static) is the correct countermeasure against a packed APK, since the protection is specifically designed to defeat static unpacking.
- Unencrypted data in shared_prefs/SQLite/external storage is exposed at the logical-acquisition tier via `adb backup` unless the app opts out.

Critical Terms

Term	Definition
ProGuard	Android SDK's built-in code shrinker/obfuscator, renaming identifiers and removing unused code.
Packer	A tool that encrypts/wraps an APK's real payload, decrypting it only at runtime to defeat static analysis.
allowBackup	An AndroidManifest.xml attribute; if not explicitly disabled, permits `adb backup` to extract an app's data.

Worked Scenario Question

1. **Q1.** *After packing your Lab-5 APK, `baksmali` fails to produce meaningful output from its classes.dex. Explain why this happens and name the correct next step in analysis.*

Model Answer: A packer replaces or encrypts the real classes.dex payload, so the classes.dex directly visible in the APK archive is often just a small loader stub whose job is to decrypt and load the true payload into memory at runtime — disassembling this stub yields little useful information about the app's real logic. The correct next step is dynamic analysis: run the packed app in a controlled/instrumented environment (emulator + Frida/proxy) and inspect its

behaviour or memory at runtime, since the real payload only exists in a readable form after the packer's runtime unpacking routine executes.

Note: *"Static analysis failed against a packed sample — what next?" always resolves to dynamic analysis; this pairing (Part VIII/IX vs Part X) is one of the most tested theory links in the course.*

PART X: DYNAMIC ANALYSIS: PROXIES, FRIDA, XPOSED, AND DROZER

Track	Source
Hacking/RE	Hacking Android, Ch. 4, 8

Overview

Dynamic analysis observes an application while it runs rather than reading its code offline, and is the natural complement (not replacement) to the static workflow in Parts VIII–IX. An intercepting proxy (Burp Suite, mitmproxy) configured on the device's Wi-Fi/APN settings, with its CA certificate trusted on the test device, reveals HTTP(S) traffic — destinations, parameters, and any credentials sent in the clear. Where an app implements SSL/TLS certificate pinning, a proxy alone is defeated unless the pinning is bypassed at runtime (e.g. via a tool such as AndroidSSLTrustKiller, or a Frida script that hooks the pinning check).

Runtime instrumentation frameworks go further: Cydia Substrate, the Xposed Framework, and Frida allow hooking and modifying method behaviour on a running app without altering its installed APK — useful for bypassing checks, dumping decrypted in-memory data (relevant against packed apps, Part IX), or tracing execution flow. Drozer is a dedicated Android security-assessment framework: it can enumerate an app's exported attack surface (activities, services, receivers, content providers) and includes modules to actively exploit them — for example, running SQL injection against a vulnerable exported Content Provider directly from Drozer's console.

Key Concepts (Compressed)

- Proxy interception (Burp/mitmproxy) requires trusting the tool's CA certificate on the test device to see decrypted HTTPS traffic.
- Certificate/SSL pinning defeats plain proxy interception; it must be bypassed at runtime (e.g. AndroidSSLTrustKiller, Frida hook) first.
- Cydia Substrate, Xposed, and Frida all enable runtime hooking/instrumentation of a running app without modifying its installed APK.
- Drozer both enumerates an app's exported attack surface AND actively exploits it (e.g. Content Provider SQL injection) from one console.

Critical Terms

Term	Definition
Certificate/SSL pinning	An app-level check that rejects any TLS certificate other than a specifically expected one, defeating generic proxy interception.
Frida	A dynamic instrumentation toolkit for hooking and modifying running application code at runtime, across platforms.
Xposed Framework	A framework for hooking and modifying Android app/system behaviour at runtime via installed modules.
Drozer	An Android security-assessment framework that enumerates and can actively exploit an app's exported components.

Practical Angle

```
# Content Provider SQL injection reconnaissance via Drozer
dz> run app.provider.info -a <package>
```

```
dz> run app.provider.query content://<authority>/<path> --selection "'"
dz> run app.provider.query content://<authority>/<path> --projection "* FROM sqlite_master--"
```

Worked Scenario Question

1. **Q1.** *Your Burp Suite proxy shows no traffic from a target app even though the CA certificate is correctly installed on the test device. What is the most likely explanation, and what is the appropriate next step?*

Model Answer: The app most likely implements SSL/TLS certificate pinning, which rejects the proxy's certificate even though it is trusted at the OS level, since pinning checks the certificate against a value embedded in the app itself rather than relying on the system trust store. The appropriate next step is to bypass pinning at runtime — for example using a tool such as AndroidSSLTrustKiller, or a Frida script that hooks and neutralises the app's pinning-check method — before re-attempting proxy interception.

***Note:** "No traffic despite a correctly installed CA cert" is the standard exam signature for certificate pinning — answer names pinning explicitly, not just 'network issue.'*

PART XI: ATTACKING APP COMPONENTS AND SERVER-SIDE INTERFACES

Track	Source
Hacking/RE	Hacking Android, Ch. 6–7

Overview

Building on the component model in Part VI, any Activity, Service, Broadcast Receiver, or Content Provider marked exported (explicitly, or implicitly via an intent-filter) becomes reachable by other apps on the device, including malicious ones with no special permissions. Static tools such as QARK automate scanning an app's manifest and code for such exposures. Content Providers are a particularly common SQL-injection surface, since many implementations pass client-supplied selection/projection strings into an underlying SQL query without adequate sanitisation — reachable and testable directly from adb shell's `content query` command or from Drozer (Part X).

Server-side attacks target the backend an app talks to: setting up a proxy (via APN or Wi-Fi settings) to observe and tamper with API requests, then probing for classic web-style vulnerabilities (injection, broken authentication/authorization, insecure direct object references) in the mobile app's API endpoints — the same server-side risk classes as traditional web applications, just reached through a mobile client instead of a browser.

Key Concepts (Compressed)

- Any exported component (explicit or via intent-filter) is reachable by other apps without special permission by default.
- Content Provider SQL injection is testable via adb shell `content query` or via Drozer's provider modules.
- QARK automates static detection of exported/exposed component risks in an app's manifest and code.
- Server-side attacks on a mobile app's backend mirror classic web vulnerability classes (injection, auth/authz flaws, IDOR).

Critical Terms

Term	Definition
Intent-filter	A manifest declaration describing which implicit intents a component can respond to; can implicitly expose a component.
QARK	Quick Android Review Kit — a static-analysis tool for finding security issues in Android app manifests/code.
IDOR	Insecure Direct Object Reference — a server-side flaw where an identifier can be manipulated to access another user's data.

Worked Scenario Question

1. **Q1.** *Using `adb shell content query --uri content://com.example.app.provider/users --projection "* FROM sqlite_master--"`, you receive table names you did not request. What vulnerability class does this confirm, and what is the underlying cause?*

Model Answer: This confirms SQL injection in the app's exported Content Provider. The underlying cause is that the provider concatenates the client-supplied projection parameter directly into its SQL query without validating or parameterising it, allowing an attacker to break out of the intended query structure (here, commenting out the rest of the original query with `--` and substituting a query against `sqlite_master` to enumerate the database schema).

Note: *Recognise the `--` comment-injection pattern and `sqlite_master` schema-enumeration trick — both are recurring exam signatures for Content Provider SQLi.*

PART XII: ANDROID MALWARE ANALYSIS AND DEVICE-LEVEL ATTACKS

Track	Source
Hacking/RE	Hacking Android, Ch. 9–10

Overview

Malware triage combines the static (Parts VIII–IX) and dynamic (Part X) skills covered earlier: reading AndroidManifest.xml for implausible permission combinations, searching disassembled code for suspicious API usage (e.g. DexClassLoader or other reflective/dynamic code loading, hard-coded IPs/URLs), and then, in a fully isolated network environment, observing runtime behaviour to confirm intended command-and-control (C2) destinations, SMS-stealing behaviour, or reverse-shell activity. All of this work must occur on an isolated, host-only network segment with no route to the live internet, since executing live malware samples with real network access risks establishing an actual C2 channel or propagating the sample.

Device-level (rather than app-level) attacks include bypassing screen locks: pulling and cracking the SHA1-hashed gesture.key file from a device via adb to recover a pattern lock, exploiting known CVEs (e.g. CVE-2013-6271) to bypass a lock screen without cracking any hash at all, and various physical/USB-vector attacks. These matter forensically because a compromise at the device/OS level produces a very different evidentiary footprint than an app-level compromise, and may require a different acquisition strategy altogether.

Key Concepts (Compressed)

- Malware triage = static indicators (manifest permissions, DexClassLoader/reflection, hard-coded network destinations) + isolated dynamic observation.
- Never run a malware sample with live internet access — isolated/host-only network segment only, to avoid establishing real C2 or spreading the sample.
- gesture.key holds a SHA1 hash of the pattern lock, recoverable via adb pull and offline cracking.
- CVE-2013-6271 is a named exam-relevant example of a screen-lock bypass that does not require cracking any hash at all.
- A device-level compromise (vs an app-level one) changes both the forensic footprint and the appropriate acquisition strategy.

Critical Terms

Term	Definition
DexClassLoader	An Android API for loading additional/external dex code at runtime; suspicious when found in static analysis of a sample.
C2 (Command-and-Control)	A remote server a piece of malware contacts to receive instructions or exfiltrate data.
gesture.key	A file on Android devices storing a SHA1 hash of the user's pattern-lock gesture.
CVE-2013-6271	A documented Android vulnerability enabling a lock-screen bypass without needing to crack the pattern/password hash.

Practical Angle

```
$ adb pull /data/system/gesture.key
# offline: crack the SHA1 hash to recover the pattern

$ unzip -d sample_ex sample.apk && java -jar baksmali.jar sample_ex/classes.dex
$ grep -R "DexClassLoader" sample_ex/out/  # flag dynamic code loading
```

Worked Scenario Question

1. **Q1.** *Static analysis of a suspicious 'flashlight' app shows it requests SEND_SMS and RECEIVE_BOOT_COMPLETED permissions, and its disassembled code contains a DexClassLoader call referencing a URL. What is your assessment, and what is the safe next step?*

Model Answer: This is a strong static indicator of malicious behaviour: a flashlight app has no legitimate need for SMS-sending or boot-persistence permissions, and a DexClassLoader call pulling code from a remote URL suggests the app downloads and executes additional, possibly malicious, code at runtime rather than shipping its full logic in the visible APK — consistent with an SMS-stealer or dropper pattern. The safe next step is dynamic observation strictly on an isolated, host-only network segment with no live internet route, recording any attempted outbound connections (which should fail/hang safely) as further evidence of intended C2 behaviour, rather than analysing it on a live-networked device.

Note: *Permission/purpose mismatch (e.g. flashlight app + SMS permission) combined with DexClassLoader is the single most common 'spot the malware indicator' exam pattern across both books.*

PART XIII: THE OWASP MOBILE TOP 10 (2014) IN FULL

Track	Source
Hacking/RE	Hacking Android, Ch. 4

Overview

The OWASP Mobile Top 10 (2014 edition, as referenced in Hacking Android) is the standard risk-ranking framework for mobile application security, and functions in mobile app assessments the way the OWASP Web Top 10 does for web applications. Every practical attack covered elsewhere in this guide — exported Content Provider SQL injection, unencrypted shared_prefs data, defeated certificate pinning, reflective malware loading — maps onto one or more of these ten categories, so being able to name the correct category for a given finding is itself a frequently tested skill, independent of the technical detail of the attack.

Key Concepts (Compressed)

- M1 Weak Server-Side Controls — backend API flaws (auth, injection, IDOR) reached through the mobile client.
- M2 Insecure Data Storage — sensitive data left unencrypted in shared_prefs, SQLite, or external storage.
- M3 Insufficient Transport Layer Protection — missing/weak TLS, or TLS that is easily intercepted.
- M4 Unintended Data Leakage — sensitive data exposed via logs, backups, clipboard, or caches.
- M5 Poor Authorization and Authentication — weak session/credential handling on-device or server-side.
- M6 Broken Cryptography — misuse of encryption (hard-coded keys, weak algorithms, poor key storage).
- M7 Client-Side Injection — injection flaws reachable via the app's own components (e.g. Content Provider SQLi).
- M8 Security Decisions via Untrusted Inputs — trusting Intent extras or IPC data without validation.
- M9 Improper Session Handling — tokens/sessions that don't expire or are reused insecurely.
- M10 Lack of Binary Protections — absence of obfuscation/anti-tamper/root-detection controls (directly ties to Part IX).

Critical Terms

Term	Definition
M2 – Insecure Data Storage	Sensitive data (tokens, credentials, PII) stored without encryption on-device.
M4 – Unintended Data Leakage	Sensitive data unintentionally exposed via logs, caches, backups, or the clipboard.
M7 – Client-Side Injection	Injection vulnerabilities reachable through an app's own exposed components.
M10 – Lack of Binary Protections	Absence of obfuscation, anti-tamper, or root/jailbreak-detection controls in a shipped app.

Worked Scenario Question

1. **Q1.** *The credential-logging exercise from Part VIII (writing username/password to Logcat) is later found in a real production app. Which two OWASP Mobile Top 10 categories does this best map onto, and why two rather than one?*

Model Answer: M4 (Unintended Data Leakage) is the primary mapping, since Logcat output is a classic unintended-leakage channel — other apps with log-read permission, or anyone with physical/adb access, can read plaintext credentials never meant to leave the login flow. M2 (Insecure Data Storage) is also defensible as a secondary mapping if the same credentials are simultaneously being persisted insecurely elsewhere (e.g. cached in shared_prefs) — in general, a single finding can validly map onto more than one Top 10 category when it has multiple distinct negative consequences, and an exam answer that names the strongest single fit (M4 here) while acknowledging a plausible second is stronger than one that forces an artificial single choice.

Note: *When a question asks 'which OWASP category,' check whether the finding has a secondary consequence too — acknowledging a reasonable second mapping (briefly) often scores higher than a rigid single answer.*

PART XIV: LEGAL, ETHICAL, AND PROFESSIONAL STANDARDS DEEP-DIVE

Track	Source
Forensics	Mobile Forensic Investigations, Ch. 2, 5–6

Overview

Beyond the mechanics of seizure (Part I), examiners operate under professional codes of ethics and certification standards (IACIS, ISFCE) that govern conduct independent of any specific technique: maintaining objectivity, avoiding conclusions beyond what the evidence supports, disclosing limitations and uncertainty rather than overstating findings, and preserving the defensibility of the whole process through documentation. Ethical failures — exceeding the scope of a warrant, working outside one's demonstrated competence, or failing to disclose a tool's known limitations — can invalidate an otherwise technically sound examination just as thoroughly as a chain-of-custody break can.

A recurring exam theme is distinguishing a technical limitation (e.g. 'logical acquisition cannot recover deleted data') from an ethical/procedural failure (e.g. 'the examiner claimed certainty the tool's documentation did not support'). Both can produce the same outcome — an incomplete or successfully challenged case — but they call for very different remediation in an answer.

Key Concepts (Compressed)

- IACIS and ISFCE certification/ethics codes govern examiner conduct, not just technique.
- Overstating certainty beyond what a tool/method can support is an ethical failure distinct from a technical limitation.
- Warrant/consent scope violations can invalidate evidence regardless of how well the technical acquisition itself was performed.
- Disclosure of limitations (what a method could NOT recover) is part of professional/ethical obligation, not merely good practice.

Critical Terms

Term	Definition
Scope of authority	The legally defined boundary (warrant, consent, exigent circumstance) within which a search/seizure is lawful.
Professional code of ethics	IACIS/ISFCE-style standards governing examiner objectivity, competence, and disclosure obligations.
Overstatement of findings	Presenting a conclusion with more certainty than the underlying method/evidence actually supports — an ethical, not just technical, failure.

Worked Scenario Question

1. **Q1.** *An examiner testifies 'this phone definitely had no messaging activity in the relevant period' based solely on a logical acquisition that returned no messages. A defence expert challenges this. Identify BOTH the technical and the ethical problem with the original testimony.*

Model Answer: Technical problem: a logical acquisition cannot see data outside the OS's live API surface (deleted messages, app-excluded backup data — Part II), so 'no messages found' is not equivalent to 'no messaging activity occurred'; the correct technical statement is 'no messages were recoverable via this acquisition tier.' Ethical problem: testifying to a 'definite' absence of activity overstates the certainty the method can actually support, which is a

professional/ethical failure independent of the technical limitation — the examiner should have disclosed the acquisition tier's known limitations as part of the original finding rather than presenting an unqualified conclusion.

Note: *Whenever a scenario gives you an over-confident conclusion, look for TWO separate things to name: the technical reason the conclusion is wrong, AND the professional/ethical duty that was breached by stating it so confidently.*

PART XV: ACQUISITION TIERS AND THE COMMERCIAL TOOL LANDSCAPE, COMPARED

Track	Source
Forensics	Mobile Forensic Investigations, Ch. 8–10 (comparative synthesis)

Overview

Exam questions frequently ask you to compare rather than simply define — e.g. 'why choose file-system over logical here?' This unit consolidates the acquisition-tier comparison from Part II into a single reference view, alongside the commercial tool landscape referenced across the forensics material (Cellebrite UFED, Magnet AXIOM, Oxygen Forensics), so you can quickly match a case constraint (device state, time pressure, legal risk tolerance) to the appropriate tier and, where relevant, the appropriate tool category.

Key Concepts (Compressed)

- Logical: fastest, lowest risk, recovers only live/accessible data — use when speed and low intrusiveness matter most and deleted-data recovery is not required.
- File-system: recovers more residual/cached data (including WAL/journal artefacts) at moderate risk/complexity — the common middle-ground choice.
- Physical: highest recovery potential (including deleted/unallocated data) but hardest to achieve on modern encrypted devices and carries the highest risk of evidence alteration if mishandled.
- Commercial suites (Cellebrite, Magnet, Oxygen) each have their own device-support matrices and parsing quirks — this is exactly why toolbox cross-validation (Part II) is standard practice rather than optional.

Critical Terms

Term	Definition
Device-support matrix	A vendor's documented list of which devices/OS versions a given acquisition method supports — varies by tool and changes frequently.
Cloud extraction tools	Vendor-specific modules (e.g. Cellebrite UFED Cloud Analyzer, Oxygen Forensics Cloud Extractor, Magnet AXIOM Cloud) for pulling cloud-synced data directly, given appropriate credentials/legal authority.

Worked Scenario Question

1. **Q1.** *You have limited time before a device's battery is expected to die, no charger compatible with the device is available, and the case requires only contacts and recent call logs — not deleted data. Which acquisition tier do you choose, and why?*

Model Answer: Logical acquisition, because it is the fastest tier to execute, the case's data requirements (contacts, recent call logs) sit entirely within what logical extraction exposes via the OS's live APIs, and there is no requirement here to recover deleted data that would justify the added time and technical risk of a file-system or physical acquisition under a hard power-loss deadline.

Note: *Tier-selection scenarios are testing whether you match the CASE REQUIREMENT (not just 'get the most data possible') to the tier — don't default to 'always choose the deepest tier' when the case doesn't need it.*

PART XVI: DEEP-DIVE: EXPLOITING A VULNERABLE CONTENT PROVIDER

Track	Source
Hacking/RE	Hacking Android, Ch. 7–8 (expanded worked example)

Overview

This unit expands Part XI into a full worked multi-step example, since Content Provider SQL injection is one of the most exam-favoured practical attack chains: it combines the component/manifest theory of Part VI, the exported-component risk of Part XI, and the tool usage of Drozer/adb from Part X into a single reasoning chain. The attack proceeds in three stages: (1) reconnaissance — confirm the provider is exported and identify its content:// authority/path; (2) probing — send a malformed selection/projection argument (e.g. a lone single-quote) and observe whether the app crashes or returns a database error, which confirms the input reaches raw SQL; (3) exploitation — craft a projection/selection payload that alters the query's logic (e.g. commenting out the remainder of the intended query and substituting a query against sqlite_master) to enumerate schema or exfiltrate data beyond what the provider was designed to expose.

Key Concepts (Compressed)

- Stage 1 (Recon): confirm exported="true" and no restrictive permission in the manifest; identify the content:// URI.
- Stage 2 (Probing): a single quote or unbalanced SQL syntax in a query parameter causing an error/crash confirms unsanitised input reaches SQL.
- Stage 3 (Exploitation): comment-out (`--`) plus a substitute clause against sqlite_master is the standard schema-enumeration payload pattern.
- The same three-stage pattern (recon → probe → exploit) generalises to most injection-class vulnerabilities, not just Content Providers.

Critical Terms

Term	Definition
Reconnaissance stage	Confirming a component is reachable/exported before attempting any exploitation.
Probing stage	Sending a deliberately malformed input to confirm (not yet exploit) that a vulnerability class is present.
sqlite_master	SQLite's built-in schema table, commonly targeted by injection payloads to enumerate a database's structure.

Practical Angle

```
# Stage 1: Recon
dz> run app.provider.info -a <package>
# Stage 2: Probe (a lone quote should trigger a SQL error if unsanitised)
dz> run app.provider.query content://<authority>/<path> --selection "'"
# Stage 3: Exploit (enumerate schema)
dz> run app.provider.query content://<authority>/<path> --projection "* FROM sqlite_master--"
```

Worked Scenario Question

1. **Q1.** *During Stage 2 probing, the target Content Provider returns a generic empty result instead of an error when sent a lone single-quote. Does this rule out SQL injection?*

Model Answer: Not conclusively. An empty result rather than a crash/error can indicate either (a) the input is properly sanitised/parameterised and the vulnerability is genuinely absent, or (b) the app has broad exception handling that silently swallows the underlying SQL error and returns an empty set regardless of cause. The correct next step is to try a payload designed to produce an observable difference in behaviour rather than an error (e.g. a boolean-based payload comparing a true vs. false condition and checking for a difference in the returned row count) before concluding the provider is not vulnerable.

Note: *The 'no visible error ≠ not vulnerable' distinction is a classic exam trap — always mention error-suppression as a competing explanation before ruling out injection.*

PART XVII: ANDROIDMANIFEST.XML AND THE PERMISSION MODEL

Track	Source
Hacking/RE	Hacking Android, Ch. 3–4 (synthesis)

Overview

AndroidManifest.xml is the single most information-dense file for both a security assessor and a malware analyst: it declares every component (Activity, Service, Receiver, Provider), whether each is exported, what permissions the app requests and what protection level (normal, dangerous, signature) those permissions carry, and the minimum/target SDK versions that govern which runtime security behaviours apply (e.g. runtime permission prompts only apply from a given API level onward). A mismatch between an app's declared purpose and its requested permission set is one of the fastest, highest-value triage signals available — both for a security tester scoping an assessment and for a forensic examiner triaging a suspect device for malicious apps.

Key Concepts (Compressed)

- Protection levels: normal (granted automatically), dangerous (requires user grant/runtime prompt), signature (only apps signed with the same certificate).
- minSdkVersion/targetSdkVersion in the manifest determine which runtime permission behaviours and platform protections actually apply to the installed app.
- Exported components + declared permissions together define the app's real external attack surface — read both, not just one.
- Permission/purpose mismatch (e.g. a wallpaper app requesting SEND_SMS) is a fast, high-value red flag in any manifest review.

Critical Terms

Term	Definition
Protection level	The tier (normal/dangerous/signature) governing how an Android permission is granted.
minSdkVersion / targetSdkVersion	Manifest attributes controlling the minimum supported and behaviourally-targeted Android API levels.
Signature permission	A permission granted only to apps signed with the same certificate as the app that declared it.

Worked Scenario Question

1. **Q1.** *An app targets a very old targetSdkVersion despite being newly published. What security-relevant inference can you draw, and why does it matter for both a security tester and a forensic examiner?*

Model Answer: Targeting an old SDK level can mean the app does not benefit from newer platform security defaults (e.g. runtime permission prompts, network security config restrictions, tighter default export behaviour for components), effectively behaving less securely than a modern-targeting app even on the same device/OS version. For a security tester, this widens the practical attack surface to test. For a forensic examiner, it can mean components assumed to be safely non-exported by modern defaults were, in fact, exported on this specific app, which is directly relevant when assessing how a compromise could have occurred.

Note: *targetSdkVersion questions are testing whether you know that Android's default security behaviour is NOT constant across all apps on the same device — it depends on what each app itself targets.*

PART XVIII: IOS VS ANDROID: FORENSIC ACQUISITION COMPARED

Track	Source
Forensics	Mobile Forensic Investigations, Ch. 13 (comparative synthesis)

Overview

Although this course's practical work centres on Android, exam scenarios frequently ask you to reason about iOS by comparison, since the underlying acquisition-tier concepts (Part II, Part XV) transfer directly even though the specific mechanisms differ. Android's more open ecosystem generally offers more acquisition paths (ADB-based logical/file-system extraction across many OEMs) but device-specific fragmentation complicates tool support. iOS's tightly closed ecosystem means fewer acquisition paths overall, but the AFU/BFU distinction (Part III) is the single most decisive factor: an AFU device with a trusted pairing or known backup password often permits a full file-system-level extraction via established iOS acquisition methods, while a BFU device without either is frequently limited to whatever data class-based encryption exposes before first unlock — often very little.

Key Concepts (Compressed)

- Android: more OEM fragmentation, generally more acquisition paths available via ADB across device variants.
- iOS: fewer acquisition paths overall, but AFU + trusted pairing/backup password can unlock much deeper extraction than BFU alone.
- The underlying acquisition-tier logic (logical/file-system/physical, and their trade-offs) transfers across both platforms even though the specific tools/mechanisms differ.
- For both platforms, the device's unlock/trust state at the moment of seizure is often more decisive than which specific tool is used.

Critical Terms

Term	Definition
Trusted pairing (iOS)	A cryptographic relationship established between an iOS device and a computer it has been unlocked and confirmed to trust, enabling deeper data access.
Data class-based encryption (iOS)	iOS's per-file encryption scheme where different data classes become accessible at different points in the boot/unlock lifecycle.

Worked Scenario Question

1. **Q1.** *You have two seized devices from the same suspect: an Android phone in an AFU state with USB debugging already enabled, and an iPhone in a BFU state with no known passcode or prior trusted-computer pairing. Which do you expect to yield significantly more data, and why?*

Model Answer: The Android phone, because AFU state combined with USB debugging already enabled removes two of the most common acquisition blockers at once, permitting at least a logical (and likely file-system) extraction immediately. The iPhone in a BFU state with no known passcode and no existing trusted pairing sits at the most restrictive end of iOS's encryption lifecycle, where the achievable data class exposure before first unlock is typically minimal — without a passcode or an existing trust relationship, meaningfully deeper access is unlikely without exceptional/specialist techniques outside standard tooling.

Note: *iOS/Android comparison questions almost always hinge on the SAME variable: is the device already unlocked/trusted, or not? Lead your answer with that, not with a tool-name comparison.*

PART XIX: CAPSTONE CASE STUDY: INVESTIGATING A REPACKAGED, TROJANISED APPLICATION

Track	Source
Both / Capstone	Synthesis across all sources

Overview

This unit walks a single, fully worked case end-to-end, deliberately combining every prior part, because your exam's scenario/application format is most likely to present one connected situation rather than isolated topic questions. Case: a company's internal Android app was found, on a seized employee handset, to be sending SMS messages without user awareness. The investigation must move through: legal/seizure (Part I, XIV), acquisition tier selection (Part II, XV), comparative static analysis of the clean vs. suspect APK (Part VIII), OWASP Top 10 mapping of the finding (Part XIII), SQLite/shared_prefs correlation on the device (Part IV, XI/XVI), and a properly structured presentation of findings (Part V).

Key Concepts (Compressed)

- A real case rarely tests one topic in isolation — practice moving fluidly from legal/seizure reasoning, to acquisition-tier choice, to static/dynamic analysis, to reporting, within one answer.
- Always state WHICH acquisition tier you are choosing and WHY before describing what you found with it.
- When comparing a clean vs. suspect APK, lead with the smali/manifest DIFF, not a restatement of the whole disassembly.
- Close a case-study answer with a plain-language conclusion, exactly as Part V requires for real reporting.

Critical Terms

Term	Definition
End-to-end case reasoning	Moving coherently through seizure → acquisition → analysis → reporting within a single connected answer, rather than treating each as an isolated fact.

Worked Scenario Question

1. **Q1.** *Walk through, in order, the five investigative steps you would take for the SMS-trojan case above, naming the specific technique from this guide at each step.*

Model Answer: (1) Seizure & legal scoping (Part I/XIV): confirm authority to examine the company device, isolate it from networks to prevent remote tampering, and document its state. (2) Acquisition (Part II/XV): given the case needs both the installed APK and on-device app data, choose a file-system-tier acquisition (or logical, if the app does not exclude itself from backup) to capture both the APK and its data directory. (3) Comparative static analysis (Part VIII): disassemble both the known-clean and the suspect APK with baksmali and diff the smali output to isolate the injected SMS-sending logic, checking the manifest for an added SEND_SMS permission not present in the clean build. (4) Artefact correlation (Part IV/XI/XVI): examine the app's SQLite databases and shared_prefs for any logged recipient numbers or message content, and confirm whether an exported component was the injection/trigger point. (5) Reporting (Part V): produce an executive summary stating plainly that the installed app was modified to send SMS messages without user awareness, supported by the diffed code and device artefacts, escalating full technical detail (exact smali diff, hashes, command log) to appendices.

Note: *Multi-step 'walk me through the investigation' questions are graded on ORDER and on naming the RIGHT TECHNIQUE per step — rehearse this five-step shape until it's automatic.*

PART XX: COMMON EXAM TRAPS AND DISTRACTORS

Track	Source
Both	Synthesis / examiner-pattern analysis

Overview

Certain wrong-answer patterns recur across both tracks because they exploit genuinely easy-to-confuse pairs of concepts. This unit exists purely to pre-empt them; each trap below has already appeared, in substance, in an earlier scenario — this is the consolidated 'don't fall for it' list.

Key Concepts (Compressed)

- Trap 1: Treating ICCID and IMSI as interchangeable — they identify the card and the subscriber respectively, not the same thing.
- Trap 2: Assuming 'no data found' by a tool means 'no data existed' — always consider parsing failure, acquisition-tier limitation, or backup exclusion first.
- Trap 3: Confusing invoke-static with invoke-virtual for a static API like Log.d — this is a bytecode verification error waiting to happen, not a stylistic choice.
- Trap 4: Believing ProGuard 'protects' an app fully — it renames identifiers but leaves strings/resources and overall control-flow shape inspectable.
- Trap 5: Trying more static analysis against a packed sample instead of switching to dynamic analysis — packing is specifically designed to defeat static disassembly.
- Trap 6: Assuming a locked/signature-protected exported component is a meaningful risk for an arbitrary third-party attacker — it generally is not, unless the signing key itself is compromised.
- Trap 7: Zipaligning before signing — the documented order is sign, then align, because signing can alter byte offsets.
- Trap 8: Treating an empty result (no error) from an injection probe as proof of 'not vulnerable' — exception handling can silently suppress the underlying error.
- Trap 9: Overstating certainty in testimony/reporting beyond what an acquisition tier or tool can actually support — a professional/ethical failure, not just an imprecise word choice.
- Trap 10: Powering off an unlocked (AFU) device 'to be safe' — this can push it into a harder-to-access BFU encryption state.

Critical Terms

Term	Definition
Distractor	An exam answer option or framing designed to be plausible but incorrect, usually by conflating two related concepts.

Worked Scenario Question

1. **Q1.** *Look back across this guide: which single trap above appears, in some form, in the greatest number of different units and scenario questions?*

Model Answer: Trap 2 — mistaking an absence of data in a tool's output for an absence of the underlying activity/data — recurs the most: it appears in the acquisition-tier discussion (Part II), the SQLite WAL/journal discussion (Part IV), the toolbox-forensics discussion (Part II), the reporting/testimony discussion (Part V, XIV), and the general scenario bank. This repetition is intentional — it is the single highest-value habit to build before the exam: whenever a scenario

says a tool 'found nothing,' your first move should be to ask what the tool COULD NOT have seen, not to accept the absence at face value.

Note: *If you remember only one meta-lesson from this entire guide, make it this: 'absence of evidence in a tool's output is not evidence of absence' — it is the most frequently rewarded reflex across both tracks.*

PART XXI: FEATURE PHONES, BLACKBERRY, AND WINDOWS PHONE: LEGACY DEVICE FORENSICS

Track	Source
Forensics	Mobile Forensic Investigations, Ch. 12 (expanded)

Overview

Legacy device classes remain forensically relevant well past their commercial peak, since older hardware persists in low-cost markets, as backup/burner devices, and in older ongoing cases. Feature phones typically lack a general-purpose file system accessible via standard tools, so extraction often relies on vendor-specific cable protocols or, for the oldest devices, physical/chip-level techniques rather than anything resembling ADB. BlackBerry devices historically layered strong device-level encryption and, for enterprise-managed units, an additional layer of Mobile Device Management (MDM)/BES (BlackBerry Enterprise Server) policy control, meaning acquisition sometimes depends as much on obtaining enterprise-side cooperation as on the device itself. Windows Phone, while short-lived, has its own distinct file-system layout and backup mechanisms distinct from both Android and iOS, and dedicated forensic tool support for it is comparatively limited — examiners often fall back on more manual, vendor-documentation-driven techniques.

Key Concepts (Compressed)

- Feature phones frequently require vendor-specific cable/protocol extraction rather than a universal tool.
- BlackBerry's enterprise-managed devices may require BES/MDM-side cooperation in addition to, or instead of, direct device acquisition.
- Windows Phone has materially less mature third-party forensic tool support than Android/iOS — expect more manual technique.
- 'Legacy' does not mean 'forensically irrelevant' — older devices still appear in live casework and exams test awareness of their distinct constraints.

Critical Terms

Term	Definition
BES (BlackBerry Enterprise Server)	Enterprise management infrastructure for BlackBerry devices, sometimes a necessary acquisition avenue alongside the device itself.
Vendor-specific cable protocol	A proprietary, non-standard data-transfer protocol required for certain feature-phone extractions in place of a universal method like ADB.

Worked Scenario Question

1. **Q1.** *You are asked to acquire data from an enterprise-managed BlackBerry device, but the device itself refuses all standard connections. What alternative avenue should you pursue, and why?*

Model Answer: Pursue the organisation's BES (BlackBerry Enterprise Server) administration, since enterprise-managed BlackBerry devices are frequently controlled and backed up at the server/policy level; with appropriate legal authority, the organisation's BES-side records and backups may hold data equivalent to, or exceeding, what a direct device acquisition would recover — and may be accessible even when the physical device itself resists standard forensic connection methods.

Note: *Any BlackBerry scenario is testing whether you think beyond the physical device to the enterprise management layer — a very different mental model from consumer Android/iOS acquisition.*

PART XXII: CLOUD FORENSICS AND MULTI-DEVICE EVIDENCE CORRELATION

Track	Source
Forensics	Mobile Forensic Investigations, Ch. 3–4 (expanded)

Overview

Modern mobile evidence rarely lives on a single device in isolation: contacts, messages, photos, and app data routinely sync to cloud accounts (vendor cloud backups, third-party app cloud sync, email-linked account data), meaning the same evidentiary fact can often be independently obtained from more than one source — the device, a cloud account, or a second device logged into the same account. Vendor-specific cloud-extraction modules (Oxygen Forensics Cloud Extractor, Cellebrite UFED Cloud Analyzer, Magnet AXIOM Cloud) exist precisely to pull this account-level data directly, given appropriate credentials or legal process, independent of physical access to any specific handset.

This redundancy is valuable both offensively (as a corroboration source, Part IV/XI) and as a resilience strategy when a specific device is damaged, wiped, or otherwise inaccessible — cloud-side data may still be obtainable even when the physical device cannot be examined at all.

Key Concepts (Compressed)

- The same fact (a message, a contact, a photo) can independently exist on-device, in a cloud account, and on a second device logged into the same account — always consider all three when a specific device is unavailable or wiped.
- Cloud-extraction requires appropriate credentials or legal process, distinct from device-level acquisition authority — do not assume one authorises the other.
- Named cloud-extraction tools (Oxygen Cloud Extractor, Cellebrite UFED Cloud Analyzer, Magnet AXIOM Cloud) are vendor-specific modules for this purpose.

Critical Terms

Term	Definition
Cloud-extraction module	A vendor-specific tool component for retrieving account-level cloud-synced data independent of physical device access.
Multi-device correlation	Cross-referencing evidence appearing on more than one device/account instance to strengthen or challenge a finding.

Worked Scenario Question

1. **Q1.** *A suspect's phone was factory-reset before seizure, destroying all local data. What is your best remaining avenue for recovering the account's messages and contacts, and what must you obtain first?*

Model Answer: Pursue cloud-account extraction (via the appropriate vendor cloud-extraction tool or direct request to the service provider), since a factory reset destroys local device data but does not typically affect data already synced to a cloud account prior to the reset. Before proceeding, you must obtain proper legal authority/process specifically covering cloud-account access (a warrant/order scoped to the account, or valid consent/credentials), since authority to examine the physical device does not automatically extend to a separate cloud service.

Note: *'Device was wiped/factory-reset' scenarios are testing whether you pivot immediately to cloud-account extraction as the next logical step, while correctly flagging that separate legal authority is required.*

PART XXIII: ANDROID BACKUP INTERNALS: .AB FORMAT AND THE ANDROID BACKUP EXTRACTOR

Track	Source
Hacking/RE	Hacking Android, Ch. 5 (expanded)

Overview

`adb backup` produces a proprietary .ab archive, not a plain tar file, even though it wraps tar-formatted data internally; the .ab file begins with a specific header (including a compression flag and, optionally, encryption if a backup password was set) that must be stripped/processed before standard archive tools can read the contents. The Android Backup Extractor utility exists specifically to convert a .ab file into a standard, directly browsable tar archive, at which point normal forensic file-system examination techniques (Part IV, XI) apply to whatever the backup contains. Because `adb backup`'s scope is itself governed by each app's manifest (the allowBackup attribute, Part IX), the resulting .ab archive's completeness is only ever as good as what each installed app has chosen to expose to Android's backup mechanism — a security-conscious app may exclude itself entirely.

Key Concepts (Compressed)

- adb backup produces a .ab file: a custom header (compression/encryption flags) wrapping tar-formatted data, not a plain tar file itself.
- Android Backup Extractor converts .ab → standard tar for normal file-system-level examination.
- A backup password set at extraction time encrypts the .ab archive, requiring that password to process it afterward.
- Each app's own allowBackup manifest setting governs whether/how much of it appears in the resulting archive — completeness is not guaranteed app-by-app.

Critical Terms

Term	Definition
.ab file	The proprietary archive format produced by `adb backup`, wrapping (optionally compressed/encrypted) tar data behind a custom header.
Android Backup Extractor	A utility that converts a .ab archive into a standard tar file for normal examination.

Practical Angle

```
$ adb backup -apk -shared -all -f case001.ab
$ java -jar abe.jar unpack case001.ab case001.tar    # Android Backup Extractor
$ tar -xf case001.tar -C case001_extracted/
```

Worked Scenario Question

1. **Q1.** *You run `adb backup` successfully, but attempting to open the resulting .ab file directly with a standard archive tool fails. What is the correct next step, and why does the direct approach fail?*

Model Answer: The direct approach fails because a .ab file is not a plain tar/zip archive — it begins with a proprietary header (specifying compression and, if a backup password was set, encryption) wrapping the actual tar-formatted payload, which standard archive tools do not know how to parse. The correct next step is to process the file with the

Android Backup Extractor utility to strip the header and (if applicable) decrypt it, producing a standard tar archive that conventional tools can then open normally.

Note: *".ab file won't open in a standard archive tool" is a direct, factual recall question — always name the Android Backup Extractor by name as the fix.*

PART XXIV: MALWARE INDICATOR PATTERNS AND SIGNATURE-STYLE DETECTION LOGIC

Track	Source
Hacking/RE	Hacking Android, Ch. 9 (expanded)

Overview

Beyond the individual red flags named in Part XII, effective triage benefits from thinking in indicator PATTERNS rather than single facts, since real malware samples rarely present just one isolated red flag. A composite pattern — permission/purpose mismatch, plus reflective/dynamic code loading (DexClassLoader), plus a hard-coded or suspicious network destination, plus unusually heavy code obfuscation relative to the app's apparent simplicity — is a far stronger basis for a malicious classification than any single element alone. This 'pattern over single-fact' reasoning mirrors how signature-based detection engines work in practice: they score a weighted combination of indicators rather than triggering on one feature in isolation, precisely to reduce false positives from legitimate apps that innocently share one such feature (e.g. a legitimate app that is heavily obfuscated purely for IP-protection reasons, with no other red flag present).

Key Concepts (Compressed)

- Composite indicator patterns (permission mismatch + reflective loading + suspicious network destination + disproportionate obfuscation) are far stronger evidence than any single indicator alone.
- A single matching indicator in isolation (e.g. obfuscation alone) is consistent with entirely legitimate reasons and should not, by itself, drive a malicious classification.
- This mirrors real signature/heuristic detection engine design: weighted multi-indicator scoring, not single-feature triggers.

Critical Terms

Term	Definition
Composite indicator pattern	A combination of multiple individually-weak signals that together form strong evidence of malicious intent.
False positive	An incorrect malicious classification triggered by an innocent app that happens to share one suspicious-looking trait.

Worked Scenario Question

1. **Q1.** *A legitimate, well-known productivity app is heavily obfuscated and uses reflection internally, but requests only permissions consistent with its stated purpose and contacts only its own known, documented backend. Should this be flagged as malware based on the obfuscation and reflection alone?*

Model Answer: No. Obfuscation and reflective code use are individually common and often entirely legitimate practices (intellectual-property protection, certain library/framework internals), and in this case neither the permissions requested nor the network destinations contacted show any mismatch or suspicious indicator. Flagging this app as malicious based solely on obfuscation/reflection, absent any other corroborating red flag, would be a false positive — the correct triage conclusion is that this composite pattern does not currently meet the threshold for a malicious classification, though continued monitoring of any NEW indicator would be reasonable practice.

Note: *Single-indicator-only scenarios (usually just obfuscation, or just reflection, with nothing else suspicious) are testing whether you resist over-flagging — the correct answer is often 'not sufficient evidence alone.'*

PART XXV: SERVER-SIDE SECURITY FOR MOBILE APP BACKENDS

Track	Source
Hacking/RE	Hacking Android, Ch. 6 (expanded)

Overview

A mobile app's client-side code is only half of its attack surface; the backend API it talks to carries the same risk classes as any traditional web application — broken authentication/authorization, injection, insecure direct object references (IDOR), and excessive data exposure in API responses — just reached through a mobile client rather than a browser. Setting up a proxy (via the device's Wi-Fi or APN settings) is the standard first step to observe exactly what the app sends and expects back, after which the tester probes each endpoint methodically: does changing a user/object identifier in a request return another user's data (IDOR)? Does the API trust client-supplied fields (e.g. a 'role' or 'isAdmin' flag sent from the device) that should instead be authoritatively determined server-side (mapping to M8, Security Decisions via Untrusted Inputs, Part XIII)?

Key Concepts (Compressed)

- Mobile app backends carry the same risk classes as traditional web APIs — injection, broken auth/authz, IDOR, excessive data exposure.
- Proxy setup (Wi-Fi/APN) is the standard first step to observe and then tamper with API requests/responses for testing.
- IDOR testing: does changing an identifier in a request expose another user's data without proper authorization checks?
- Never trust client-supplied authorization/role fields as authoritative — this is a direct instance of OWASP M8.

Critical Terms

Term	Definition
IDOR	Insecure Direct Object Reference — a flaw where manipulating an identifier grants access to another user's data without proper authorization checks.
Excessive data exposure	An API returning more data in its response than the client actually needs, risking leakage of sensitive fields.

Worked Scenario Question

1. **Q1.** *While proxying a mobile app's traffic, you notice a request to `/api/user/1042/profile` returns your own profile. Changing the URL to `/api/user/1043/profile` returns a different user's full profile, including their email and phone number, with no additional authentication check. What vulnerability is this, and what should a remediated version of this endpoint do differently?*

Model Answer: This is a textbook Insecure Direct Object Reference (IDOR): the API relies solely on a client-supplied, easily guessable/incrementable identifier to determine which user's data to return, without independently verifying that the authenticated requester is actually authorized to view that specific user's profile. A remediated endpoint should perform a server-side authorization check on every request — confirming the authenticated session's own user ID matches (or has explicit permission to view) the requested profile ID — rather than trusting the URL parameter alone to gate access to sensitive data.

Note: *IDOR scenarios almost always follow this exact shape (increment/change an ID, get someone else's data) — recognise the pattern instantly and always answer with 'server-side authorization check per request,' not just 'validate input.'*

PART XXVI: DROZER: THE STANDARD ASSESSMENT WORKFLOW, END TO END

Track	Source
Hacking/RE	Hacking Android, Ch. 8 (expanded workflow reference)

Overview

Drozer recurs across Parts X, XI, and XVI because it is the single tool most likely to anchor a multi-part practical exam question: it combines reconnaissance and exploitation in one console session against a live app on a connected device/emulator. The standard workflow is: (1) install and start the Drozer agent on the target device/emulator and connect the console to it; (2) run `app.package.list` or a search to identify the target package; (3) run `app.package.attacksurface <package>` to enumerate every exported component in one step; (4) for each interesting component type, run the matching family of modules — `app.activity.*` for Activities, `app.service.*` for Services, `app.broadcast.*` for Receivers, `app.provider.*` for Content Providers; (5) escalate from information-gathering modules to exploitation modules (e.g. `app.provider.query` with an injection payload, or `app.activity.start` to launch a protected Activity directly) only once the attack surface is properly mapped.

Key Concepts (Compressed)

- Drozer's workflow is always: connect agent → identify package → attacksurface enumeration → per-component-type modules → exploitation modules, in that order.
- `app.package.attacksurface` is the single highest-value first command against any new target — it answers 'what is even reachable' before anything else.
- Each Android component family (activity/service/broadcast/provider) has its own matching Drozer module family — know the naming pattern, not just individual commands.
- Exploitation modules should only be run against components already confirmed reachable/exported by the reconnaissance modules — skipping recon wastes time on unreachable targets.

Critical Terms

Term	Definition
Drozer agent	The component installed on the target Android device/emulator that the Drozer console communicates with to run assessment modules.
app.package.attacksurface	The Drozer module that enumerates all exported components of a target package in a single command.

Practical Angle

```
$ adb install drozer-agent.apk
$ adb forward tcp:31415 tcp:31415
$ drozer console connect
dz> run app.package.list -f <keyword>
dz> run app.package.attacksurface <package>
dz> run app.activity.info -a <package>
dz> run app.activity.start --component <package> <exported.activity>
```

Worked Scenario Question

1. **Q1.** *A student runs an exploitation module (`app.provider.query` with an injection payload) as their very first Drozer command against a new target, without running attacksurface first. What is wrong with this approach, independent of whether the exploit happens to succeed?*

Model Answer: Even if the exploit happens to succeed, skipping the reconnaissance step means the student has no systematic picture of the app's full attack surface — they may miss other exported, more severely exploitable components entirely, and cannot demonstrate (for a report or exam answer) a methodical assessment process. The correct methodology is recon-before-exploitation precisely so that effort is directed at components confirmed reachable, and so that the assessment as a whole is defensible and complete rather than a single lucky guess.

Note: *Any Drozer question that skips straight to an exploitation module in the 'given' scenario is testing whether YOU still name attacksurface enumeration as the correct first step in your answer.*

PART XXVII: ENCRYPTION AND KEY MANAGEMENT FAILURES ON MOBILE

Track	Source
Hacking/RE	Hacking Android, Ch. 5–6 (expanded, broken cryptography)

Overview

Broken cryptography (OWASP M6, Part XIII) is less often about using a mathematically weak algorithm outright, and far more often about key management failures surrounding otherwise-adequate algorithms: hard-coding an encryption key directly in Java/Kotlin source (trivially recoverable via the same static disassembly workflow as Part VIII), deriving a key from a predictable/static value (e.g. the app's own package name or a fixed string) rather than a securely generated and stored secret, or storing an otherwise-correctly-generated key in an insecure location (shared_prefs in plaintext) rather than using the platform's dedicated secure key storage (Android Keystore system). An analyst reviewing an app for this class of flaw should specifically search disassembled code for cipher initialisation calls and trace backward to see where the key material actually originates.

Key Concepts (Compressed)

- Broken cryptography is usually a KEY MANAGEMENT failure, not an algorithm-strength failure — hard-coded keys, predictable key derivation, or insecure key storage.
- A hard-coded key in source is recoverable via the same static disassembly workflow used throughout this guide (Part VIII) — there is no meaningful secrecy once an attacker has the APK.
- The Android Keystore system is the platform-correct location for key material; shared_prefs/SQLite storage of raw key bytes is a red flag.
- Tracing backward from a cipher initialisation call to its key's true origin is the standard analyst technique for this vulnerability class.

Critical Terms

Term	Definition
Android Keystore system	Android's platform-provided secure storage facility for cryptographic keys, isolated from normal app-accessible storage.
Predictable key derivation	Generating an encryption key from a fixed, guessable value (e.g. a package name) rather than a securely random or properly derived secret.

Worked Scenario Question

1. **Q1.** *You find a Cipher.init() call in disassembled smali, and tracing backward shows the key byte array is built from the string "com.example.myapp" via a simple hash function. Why is this a critical finding even though the app technically does use encryption?*

Model Answer: Because the key is deterministically derived from a fixed, publicly-known value (the app's own package name, visible to anyone who downloads the APK), any attacker can independently compute the exact same key without ever needing to extract it from the app's memory or storage — the encryption provides no real confidentiality against anyone willing to reproduce the same trivial derivation. This is a more severe finding than it might first appear, precisely because 'the app uses encryption' sounds reassuring in a report but is functionally meaningless here; the correct characterisation is that the data is not meaningfully protected at all.

Note: *"The app uses encryption, so it's fine" is a trap phrase — always check where the KEY comes from before accepting that a finding is resolved.*

PART XXVIII: ADVANCED PHYSICAL ACQUISITION: CHIP-OFF AND JTAG TECHNIQUES

Track	Source
Forensics	Mobile Forensic Investigations (advanced acquisition synthesis)

Overview

When standard logical, file-system, and even software-based physical acquisition methods (Part II, XV) are unavailable — a severely damaged device, a locked device with no software exploit path, or unsupported/obsolete hardware — examiners may resort to hardware-level techniques. JTAG (Joint Test Action Group) acquisition uses a device's debug interface pins to read raw flash memory directly at the hardware level, without needing to boot the device's OS at all, making it viable even on devices with severe software-level lockouts, provided the JTAG interface itself is accessible and not disabled. Chip-off acquisition goes further still: physically desoldering the flash memory chip from the device's circuit board and reading it directly with specialised hardware, which is destructive to the device (it cannot be reassembled into a working phone afterward) and is generally treated as a last-resort technique reserved for cases where the evidentiary value justifies the destructive, irreversible nature of the method and no less invasive option remains viable.

Key Concepts (Compressed)

- JTAG: reads raw flash via the device's hardware debug interface, without booting the OS; non-destructive to the device but requires specialist equipment/access.
- Chip-off: physically removes and directly reads the flash memory chip; the most invasive/destructive technique, and irreversible.
- Both are last-resort techniques used only when software-based acquisition methods are unavailable or have failed.
- The decision to use chip-off in particular should weigh evidentiary necessity against the destructive, irreversible nature of the technique.

Critical Terms

Term	Definition
JTAG	A hardware debug interface standard; in forensics, used to read raw device flash memory directly without booting the OS.
Chip-off	Physically desoldering and directly reading a device's flash memory chip; destructive and irreversible.

Worked Scenario Question

1. **Q1.** *A severely water-damaged phone will not power on at all, and the case is a missing-persons investigation where speed and maximum data recovery both matter greatly. Is chip-off acquisition an appropriate first choice here, or should JTAG be attempted first? Justify your answer.*

Model Answer: JTAG should generally be attempted first if the device's circuit board and JTAG interface are still physically accessible and intact, since it is non-destructive and preserves the option to escalate to chip-off afterward if JTAG proves unsuccessful or the interface is inaccessible/damaged. Chip-off should be reserved as the next step if JTAG is not viable (e.g. the JTAG interface itself is damaged or the device's specific chipset does not expose a usable JTAG path), given that chip-off's destructive, irreversible nature forecloses any further non-destructive attempt once performed — in a time-and-evidence-critical case like a missing-persons investigation, preserving optionality by trying

the less destructive method first is the more defensible sequencing, not a reason to skip straight to the most invasive option 'to save time.'

Note: *"Which hardware technique first" questions are testing least-invasive-first sequencing (JTAG before chip-off), the same underlying principle as the logical-before-physical rule in Part XV.*

PART XXIX: WHEN FORENSICS MEETS A LIVE SECURITY INCIDENT

Track	Source
Both / Applied Synthesis	Synthesis across both source books

Overview

Not every mobile forensic engagement begins with a device already seized after the fact; some begin as an active security incident — a company suspects a device is currently compromised and communicating with an attacker right now. This blends the forensic discipline (Parts I–V, XIV–XV, XVIII, XXI–XXII) with the security-testing discipline (Parts VI–XIII, XVI–XVII, XXIII–XXVII): the responder must decide, often quickly, whether to isolate the device immediately (destroying the chance to observe live attacker behaviour, but stopping ongoing harm) or to observe it briefly under controlled conditions first (preserving investigative value, but allowing continued exposure). This decision mirrors, in miniature, the malware-isolation principle from Part XII/XXIV — any live observation must still occur in a controlled, isolated-network context, never on the live production network.

Key Concepts (Compressed)

- Live-incident mobile response must balance stopping ongoing harm against preserving investigative value — there is no universally correct default; the decision depends on case severity and risk tolerance.
- Any live/dynamic observation of a suspected-compromised device must occur on an isolated network, mirroring the malware-analysis isolation principle (Part XII/XXIV), even in a live incident-response context.
- Live incident response draws on BOTH tracks of this guide simultaneously — acquisition-tier reasoning (forensics) and static/dynamic analysis reasoning (hacking/RE) in the same engagement.
- Documentation discipline (Part I, V, XIV) does not relax just because the situation is live/urgent — if anything, contemporaneous documentation matters more under time pressure.

Critical Terms

Term	Definition
Live incident response	Security investigation of a device/system believed to be actively compromised, as opposed to a purely retrospective forensic examination.
Contain vs. observe trade-off	The decision between immediately isolating a compromised device (stopping harm) versus briefly observing it under control (preserving investigative value).

Worked Scenario Question

1. **Q1.** *A company's security team believes an employee's Android phone is actively exfiltrating data to an external server right now. They ask whether to immediately put the phone in Airplane Mode or to first observe its traffic for ten minutes to identify the destination. Advise them, explaining the trade-off explicitly.*

Model Answer: This is a genuine trade-off with no universally correct answer, and the advice should make that trade-off explicit rather than picking a default: immediately isolating the device (Airplane Mode) stops any further data loss right away but forecloses the chance to observe the live exfiltration destination, which could be valuable both for understanding the scope of compromise and for any subsequent legal/law-enforcement action. Briefly observing traffic (via a controlled proxy/network tap the team already controls, NOT the open production network) for a short, bounded window can capture that destination information, but extends the exposure window by that same amount of time. The recommendation should be case-severity-dependent: if the data being exfiltrated is highly sensitive/high-

volume, favour immediate isolation; if the destination/scope information is itself critical to the response (e.g. for legal action or determining what else may be compromised) and the additional exposure window is short and tightly bounded, a brief controlled observation may be justified — but this should be a deliberate, documented decision, not a default in either direction.

Note: *"Isolate now vs. observe first" questions want you to name the trade-off explicitly and make the recommendation conditional on stated case factors — a flat, unconditional answer either way scores lower than a reasoned, conditional one.*

PART XXX: EXAM TECHNIQUE: STRUCTURING A SCENARIO ANSWER FOR MAXIMUM MARKS

Track	Source
Both / Exam Skills	Guide-wide synthesis

Overview

Knowing the content is necessary but not sufficient for a scenario/application-based exam; how you STRUCTURE an answer under time pressure materially affects the mark it receives, because graders are typically working from a rubric that rewards specific, nameable steps and justifications rather than a correct-sounding paragraph of prose. This unit distils the answer SHAPE used consistently across every worked scenario and essay in this guide, so you can reproduce it reliably under exam conditions even for a topic combination you haven't seen exactly before.

Key Concepts (Compressed)

- Shape 1 — Name the concept/technique explicitly (by its correct term) before explaining it in your own words; graders reward correct terminology, not just correct reasoning described vaguely.
- Shape 2 — State WHY, not just WHAT: 'I would use file-system acquisition' scores less than 'I would use file-system acquisition, because the case requires recovering residual WAL data that a purely logical extraction cannot reach.'
- Shape 3 — For multi-step scenarios, answer IN ORDER and number your steps; order itself is often part of the rubric (see the capstone case study, Part XIX).
- Shape 4 — Where a scenario is genuinely a trade-off (Part XXIX, and several Traps in Part XX), name BOTH sides explicitly before giving a conditional recommendation, rather than picking one side silently.
- Shape 5 — Close with the honest limitation or the plain-language translation where the question calls for it (reporting-style questions, Part V/XIV) — don't just stop at the technical finding.

Critical Terms

Term	Definition
Rubric-aligned answering	Structuring an exam answer to explicitly surface the specific, nameable elements a marking scheme is likely to reward.

Worked Scenario Question

1. **Q1.** *Take any single scenario question from Part XXII (Scenario Bank) and rewrite your answer to it using all five shapes above, in a single paragraph, without adding any new facts beyond what the model answer already contains.*

Model Answer: There is no single 'correct' rewrite here — this is a self-practice exercise. A strong rewrite will: (1) open by naming the specific technique/concept by its correct term; (2) immediately follow with the reason/justification tied to the specific facts given in the scenario, not a generic restatement of theory; (3) if the original scenario involves more than one step, present them as an explicit numbered or clearly sequential list; (4) if the scenario contains a genuine trade-off, state both competing considerations before landing on a conditional recommendation; and (5) close with either the honest limitation of the chosen approach or a plain-language translation, if the question's framing calls for either. Compare your rewrite against the model answer's structure (not just its content) to check whether you reproduced the same SHAPE, independent of wording.

Note: *This is the one unit worth re-reading on the morning of the exam alongside Part XXXI (Rapid Revision Checklist) — content recall gets you the facts; this unit gets you the marks for using them correctly.*

PART XXXI: QUICK-REFERENCE COMPARISON TABLES

Side-by-side comparisons for the pairs/sets of concepts most often confused under exam pressure.

Acquisition Tiers Compared

Tier	Speed / Risk	Can Recover Deleted Data?	Typical Requirement
Logical	Fastest / lowest risk	No	OS-exposed APIs only (adb backup/pull)
File-System	Moderate / moderate risk	Sometimes (WAL/journal, cache remnants)	Often needs elevated/root-adjacent access
Physical	Slowest / highest risk	Yes, if achievable	Hardest on modern encrypted devices; specialist technique

ICCID vs IMSI vs LOCI vs FPLMN

Field	Identifies	Forensic Use
ICCID	The physical SIM/UICC card itself	Prove a specific card was used in a specific handset at a specific time
IMSI	The subscriber account with the network	Correlate with carrier call/data records regardless of card/handset swaps
LOCI / EF-LOCI	Last registered network cell/area	Rough geographic/network history
FPLMN	Networks that refused the SIM service	Supplementary network-history/geography indicator

Android vs iOS Acquisition (Key Variable Compared)

Platform	Most Decisive Factor	Best Case	Worst Case
Android	USB debugging already enabled + unlock state	Unlocked, debugging enabled → logical/file-system readily achievable	Locked, debugging disabled, no known passcode → very restricted
iOS	AFU vs BFU + existing trusted pairing/backup password	AFU + trusted pairing/known password → deep extraction possible	BFU, no pairing, no password → minimal data-class exposure only

OWASP Mobile Top 10 (2014) Quick Reference

Code	Risk Category	One-Line Example
M1	Weak Server-Side Controls	Backend API with broken auth/injection reached via the mobile client
M2	Insecure Data Storage	Credentials cached unencrypted in shared_prefs or SQLite
M3	Insufficient Transport Layer Protection	Session token sent over plain HTTP
M4	Unintended Data Leakage	Credentials written to Logcat (Part VIII exercise)
M5	Poor Authorization and Authentication	Weak session/credential handling on-device or server-side
M6	Broken Cryptography	Hard-coded keys or weak algorithms
M7	Client-Side Injection	Exported Content Provider SQL injection
M8	Security Decisions via Untrusted Inputs	API trusting a client-supplied 'isAdmin' flag
M9	Improper Session Handling	Tokens that never expire or are reused insecurely
M10	Lack of Binary Protections	No obfuscation/anti-tamper/root-detection at all

PART XXXII: CONSOLIDATED COMMAND & API CHEAT-SHEET

Every command/API referenced across the guide, grouped by function, for rapid last-minute recall.

Environment & ADB Basics

Command / API	What it does
adb devices	List connected devices/emulators and their connection state.
adb shell	Open an interactive shell on the connected device.
adb install app.apk / adb uninstall <pkg>	Install / remove an APK by package name.
adb pull <remote> <local> / adb push	Copy files from device to host, or host to device.
adb logcat	Stream the device's system log (used to view injected Log.d output).
adb backup -apk -shared -all -f out.ab	Logical-tier backup extraction to a .ab archive.

APK Reverse Engineering

Command / API	What it does
unzip -d dir app.apk	Unpack an APK (a ZIP archive) into a working directory.
java -jar baksmali.jar classes.dex	Disassemble classes.dex into an 'out/' directory of smali files.
java -jar smali.jar -o classes.dex out	Reassemble a smali directory back into classes.dex.
rm -r META-INF	Remove the old signing certificate directory before re-zipping.
zip -r app.zip ./* && mv app.zip app.apk	Re-package the working directory into a new APK.
keytool -genkey -keystore k.keystore -alias a -keyalg RSA -keysize 2048 -validity 10000	Generate a new signing key/keystore.
jarsigner -sigalg SHA1withRSA -digestalg SHA1 -keystore k.keystore app.apk a	Sign the rebuilt APK with the generated key.
zipalign -v 4 app.apk app-aligned.apk	Align the SIGNED apk (run this step last) to reduce runtime RAM use.

Rooting & Device-Level

Command / API	What it does
fastboot oem unlock	Unlock an OEM bootloader (device/vendor-specific).
fastboot flash recovery custom.img	Flash a custom recovery image.
Odin / Heimdall	Samsung-specific flashing tools (Windows / cross-platform) for download-mode flashing.

Command / API	What it does
adb pull /data/system/gesture.key	Retrieve the SHA1-hashed pattern-lock file for offline cracking.

Dynamic Analysis & Drozer

Command / API	What it does
run app.provider.info -a <pkg>	Drozer: enumerate a package's exported Content Providers.
run app.provider.query content://... --selection "'"	Drozer: probe a Content Provider for SQL injection.
content query --uri content://<authority>/<path>	adb shell: query a Content Provider directly.
Burp Suite / mitmproxy + device Wi-Fi proxy settings	Intercept HTTP(S) traffic; requires trusting the tool's CA cert on-device.

SQLite Forensics

Command / API	What it does
sqlite> .tables	List tables in an opened SQLite database.
SELECT name FROM sqlite_master WHERE type='table';	Enumerate tables via the schema table directly.
Check for accompanying -wal / -journal files	Recent/uncommitted data may exist only in these, not the main .db file.

PART XXXIII: SCENARIO BANK — MIXED, TIMED PRACTICE

These questions are deliberately mixed across forensics and hacking/RE topics, and out of unit order, to simulate exam unpredictability. Attempt each in ≈ 3 minutes before reading the model answer.

1. **Q1.** *A suspect's Android phone is found unlocked but the screen times out and locks while you are photographing the scene. What should you have done differently, and can the situation still be recovered?*

Model Answer: The device's screen-timeout/auto-lock setting should have been disabled (or the screen kept active by periodic non-data-altering touches, per your agency's SOP) as one of the first actions after establishing legal authority and network isolation, precisely to preserve the AFU (After First Unlock) state during documentation. It may still be recoverable: if the passcode is known/obtainable, or if a trusted USB debugging pairing already existed before the lock, some acquisition paths remain open; otherwise the device has effectively moved to a BFU-equivalent access level and the acquisition options available to you are now more limited.

2. **Q2.** *You are asked to reverse-engineer a suspicious APK that refuses to install on your test AVD with a signature-related error. What is the most likely cause?*

Model Answer: You are almost certainly trying to install an updated/modified version of an app that is already installed under the same package name but signed with a different key; Android enforces that updates to an existing package must be signed with the same certificate as the original install. Uninstalling the existing package first (`adb uninstall <package>`) before installing the new APK resolves this.

3. **Q3.** *During a logical acquisition, `adb backup` completes but the resulting .ab file appears to contain almost no data for a specific messaging app you know the suspect used heavily. What is the most likely explanation?*

Model Answer: The app most likely declares `android:allowBackup="false"` (or targets a modern API level where auto-backup is opted out by default for sensitive data), which explicitly excludes it from `adb backup`'s logical-tier extraction regardless of how much data it actually holds on-device. A file-system or physical acquisition tier would be required to reach that data directly.

4. **Q4.** *A junior colleague wants to demonstrate certificate pinning bypass directly to a courtroom jury as part of expert testimony. Is this appropriate, and what should replace it if not?*

Model Answer: A live technical demonstration of an active bypass technique is generally inappropriate for a lay jury audience — it doesn't aid comprehension and can appear needlessly technical or even alarming. This should be replaced with a plain-language explanation (as covered in the Presenting Evidence unit): describe in one or two sentences what pinning is, why the interception was necessary, and what was found, with the technical bypass method available only in a written appendix if requested.

5. **Q5.** *You find two conflicting timestamps: a SQLite `messages` table entry says 14:02, but a corresponding shared_prefs 'last_seen' value says 14:07 for what appears to be the same event. How do you proceed?*

Model Answer: Do not simply pick one value as authoritative. Investigate the source of each timestamp (device local time vs. server-synced time vs. a cached/stale preference value that may not update in real time) and look for a third, independent corroborating source (a system log entry, a network capture timestamp if available) to resolve the discrepancy. If it cannot be resolved, report both values and the discrepancy transparently rather than presenting a single, unqualified timestamp.

6. **Q6.** *A packed APK's manifest still shows only the INTERNET permission, yet dynamic analysis reveals it reading contacts and SMS at runtime. Explain this apparent contradiction.*

Model Answer: This is consistent with a packer/loader pattern where the visible, installed APK's manifest under-declares the app's true behaviour, and the real permission-requiring functionality is downloaded, decrypted, or dynamically invoked at runtime (e.g. via DexClassLoader loading a secondary payload) rather than being present in the statically declared manifest. This is exactly why dynamic analysis is required to reveal true behaviour once packing/loading is suspected — the manifest alone cannot be trusted for a hostile sample.

7. **Q7.** *Your acquisition log shows a SHA-256 hash for an extracted .ab file, but a re-hash of the same file six months later at trial produces a different value. What are your first two hypotheses, and which should you check first?*

Model Answer: First hypothesis: the file was altered (accidentally or deliberately) at some point after extraction, which would be a serious chain-of-custody failure. Second hypothesis: a copy/transfer error, corrupted storage media, or a mistaken reference to the wrong file/version produced an unintentional mismatch. Check the second hypothesis first — verify you are hashing the exact same file referenced in the original acquisition log (correct path, correct case number) and check storage media integrity — before concluding the evidence was tampered with.

8. **Q8.** *A Content Provider is exported but requires a custom signature-level permission that only the vendor's own apps hold. Is this component still a meaningful attack surface for a third-party malicious app?*

Model Answer: Generally no for a typical third-party attacker: a signature-level permission is only granted to apps signed with the same certificate as the declaring app, so an unrelated malicious app cannot obtain it and therefore cannot invoke the protected component. It remains a risk only if the signing key is compromised, or if another app legitimately signed by the same vendor is itself compromised or malicious.

9. **Q9.** *You need to determine whether a suspect's smartwatch or their (unrecovered) phone was the source of a suspicious fitness-app message. What is your investigative approach?*

Model Answer: Pursue the redundant-evidence principle: request data directly from the fitness app vendor's cloud account (via appropriate legal process), and separately look for the fitness app's companion-app cache on any other device linked to the same account (a tablet, a shared family device). Compare timestamps and device-identifier metadata across these sources to determine which device most plausibly originated the message, since the watch alone may not retain enough historical data once synced and cleared locally.

10. **Q10.** *While disassembling an app for a security assessment, you notice `invoke-virtual` is used to call what should be a static logging method. What is wrong with this smali, and what will happen if it is left uncorrected?*

Model Answer: `android.util.Log.d(String,String)` is a static method, and static methods must be invoked in smali with `invoke-static`, not `invoke-virtual` (which is for instance/virtual method calls requiring an object reference). Leaving `invoke-virtual` against a static method will cause a verification error at the Dalvik/ART bytecode verifier stage, and the app will crash (or fail to run at all) rather than silently misbehave.

11. **Q11.** *A forensic report you are reviewing states 'the phone was clean — no evidence of the messaging app was found.' The acquisition method used was a standard adb backup. What single follow-up question would you ask the original examiner?*

Model Answer: 'Did you check whether the messaging app had allowBackup disabled (or was otherwise excluded from the OS's auto-backup mechanism), and if so, what alternative acquisition tier was attempted to reach its data directly?' — since a logical `adb backup` extraction can produce a false 'no evidence' finding purely due to backup-exclusion, independent of whether the app was actually used.

12. **Q12.** *During malware triage, you identify a hard-coded IP address inside disassembled smali. Is a hard-coded IP by itself sufficient to conclude the app is malicious?*

Model Answer: No — a hard-coded IP alone is a red flag warranting further investigation, not conclusive proof of malice, since legitimate apps occasionally hard-code endpoints (analytics services, CDNs) for benign reasons. It should be combined with other indicators (permission/purpose mismatch, reflective code loading, obfuscation inconsistent with the app's stated purpose, and confirmed dynamic outbound connection attempts to that address in an isolated environment) before a malicious conclusion is drawn.

13. **Q13.** *You are asked to root a device only long enough to inspect one app's protected /data/data directory, then return the device to an unrooted, forensically 'clean' state for further examination. Is this achievable, and what should you document either way?*

Model Answer: Full reversal to a pristine unrooted state is not reliably achievable or verifiable — rooting frequently involves flashing a custom recovery/patched boot image and may leave residual artefacts (modified partitions, root-management app remnants, altered verified-boot state) even after an attempted 'unroot.' Regardless of the chosen path, you must document that the device was rooted, when, why, exactly what was inspected while rooted, and any attempt made to reverse it — full transparency about the intervention preserves defensibility even though a truly pristine state cannot be guaranteed.

14. **Q14.** *A grading rubric penalises a student for zipaligning an APK before signing it, even though the resulting APK installed successfully on the grader's test emulator. The student argues the outcome proves the order didn't matter. How do you evaluate this argument?*

Model Answer: The argument conflates 'it happened to work on one test emulator' with 'the procedure is correct.' Signing can alter the APK's internal byte layout, so alignment performed before signing is not guaranteed to survive intact — some emulators/OS versions may tolerate a resulting misalignment without visibly failing, while others (or the same device under memory pressure) may not, and any misalignment still represents unnecessary RAM inefficiency at runtime even where it doesn't cause an outright crash. The correct order (sign, then align) removes this risk entirely rather than relying on which environments happen to tolerate the mistake.

15. **Q15.** *An expert witness is asked, 'Could this evidence have been planted?' regarding a file recovered from a physical acquisition. What two things should the answer reference?*

Model Answer: The answer should reference (1) the documented chain of custody, showing the device was isolated and continuously controlled from seizure through acquisition such that no unauthorised access occurred, and (2) the acquisition's hash-verified integrity — that the extracted image's hash was recorded immediately post-acquisition and has remained unchanged since, meaning any post-acquisition planting would be independently detectable via hash mismatch.

16. **Q16.** *A student sets minSdkVersion to a very low API level 'to support more devices' for an app handling sensitive login data. What security trade-off does this introduce?*

Model Answer: A very low minSdkVersion means the app must remain compatible with, and may default to, older platform security behaviours (e.g. all permissions granted at install time rather than at runtime, weaker default component export behaviour, older TLS defaults), so the app can end up running with a materially weaker security posture on older devices than a modern-targeting app would have, even though the code itself is unchanged. Supporting more devices and maintaining strong security defaults are in tension, and a security-conscious app should isolate any low-minSdk compatibility shims rather than let the whole app's behaviour degrade to the lowest common denominator.

17. **Q17.** *You are reviewing a Drozer session log and see a tester ran `run app.package.attacksurface <package>` before any exploitation module. Why is this a sound methodological choice?*

Model Answer: This command enumerates the app's exposed attack surface (exported activities, services, receivers, providers) in one step, which is the correct reconnaissance-first approach — it lets the tester scope which specific components are actually worth targeting with exploitation modules, rather than blindly running exploitation attempts against components that may not even be exported or reachable.

18. **Q18.** *A commercial forensic tool's report includes a disclaimer stating 'results should be independently verified for evidentiary use.' A student argues this disclaimer means the tool's output should never be trusted at all. Evaluate this reasoning.*

Model Answer: The reasoning overstates the disclaimer's meaning. The disclaimer reflects standard toolbox-forensics practice (Part II) — that any single tool's output should be corroborated, not that the tool's output is inherently untrustworthy or useless. The correct response to such a disclaimer is to apply independent verification/validation before relying on a significant finding in a report, not to discard the tool's output altogether; discarding a well-regarded tool's findings without cause would itself be an unjustifiable and inefficient approach.

19. **Q19.** *During dynamic analysis, Frida successfully hooks a target method, but the hooked function's return value appears unchanged in the app's behaviour. Suggest two possible reasons.*

Model Answer: First, the hook may be attached to the wrong overload/signature of the method (Android/Java methods can be overloaded), so a different, un-hooked version of the method is actually being called at runtime. Second, the app may cache the method's result earlier in its execution flow (e.g. at app startup) and never re-invoke the hooked method again during the observed session, meaning the hook is technically working but the cached original value is what the app actually uses thereafter.

20. **Q20.** *A student claims that because ProGuard is 'free and built into the Android SDK,' there is no reason not to enable it on every release build. Is this a reasonable blanket claim?*

Model Answer: Largely reasonable as a default posture — ProGuard's shrinking/renaming carries real reverse-engineering-cost benefits at effectively no licensing cost — but not unconditionally true: enabling it without correctly scoped keep rules can break reflection-dependent libraries, serialization, or dependency-injection frameworks at runtime, so it requires testing and tuned keep rules, not a blind toggle. The claim is reasonable about cost/benefit in principle but incomplete about the engineering care required to enable it safely.

21. **Q21.** *In a forensic report, an examiner writes: 'The suspect definitely sent this message, because it appears in the app's SQLite database.' Identify the overstatement and correct it.*

Model Answer: The overstatement is asserting the SUSPECT sent it, when the database only establishes that the message exists in association with that ACCOUNT/DEVICE. Without additional corroboration (e.g. confirming who had control of the device/account at that timestamp, cross-referencing with other artefacts), the correct, defensible statement is that 'a message consistent with being sent from this account was recorded at [timestamp],' with attribution to a specific individual requiring separate corroborating evidence of who was in possession/control of the device or account at that time.

22. **Q22.** *You intercept traffic showing a mobile app sending an API request over plain HTTP (not HTTPS) containing a session token. Map this finding to the OWASP Mobile Top 10 and state the risk.*

Model Answer: This maps primarily to M3 (Insufficient Transport Layer Protection): sending a session token over unencrypted HTTP exposes it to any party able to observe network traffic (e.g. on a shared Wi-Fi network), enabling session hijacking. It also has a secondary relationship to M9 (Improper Session Handling) if the token additionally lacks expiry or reuse protections, compounding the risk beyond the transport-layer exposure alone.

23. **Q23.** *A device's Drozer assessment shows an exported Service (not Activity or Provider) with no permission requirement. Explain a plausible attack an adversary could mount against it and how it would be tested.*

Model Answer: A plausible attack is invoking the Service directly from another app (or via adb) to trigger unintended background functionality — for example, starting a Service that performs a privileged action (sending data, modifying settings) without the normal UI-driven user consent flow that would otherwise gate it. This would be tested using Drozer's `run app.service.start` (or `app.service.info`/`app.service.send`) modules to invoke the exported Service directly and observe whether the privileged action occurs without any user interaction.

24. **Q24.** *Two examiners disagree: one says physical acquisition should always be attempted first 'to maximise recoverable evidence,' the other says the least invasive tier sufficient for the case should always be chosen first. Whose position better reflects standard practice, and why?*

Model Answer: The second examiner's position better reflects standard practice. Standard acquisition methodology (Part II, XV) favours choosing the least invasive tier that meets the case's actual evidentiary requirements, because physical acquisition carries the highest risk of altering or damaging evidence and is often unnecessary — 'maximise evidence at any cost' is not the guiding principle; 'sufficient, defensible, and proportionate to the case' is.

25. **Q25.** *A student disassembles an app, finds obfuscated (ProGuard-renamed) class names like `a.b.c`, and concludes 'the source code is now unrecoverable and the analysis should stop.' Evaluate this conclusion.*

Model Answer: This conclusion is premature. ProGuard renaming affects identifiers only, not the app's actual logic, control flow, string literals, or resource files — an analyst can still trace execution logic, read any embedded string constants (API keys, URLs, log tags), and infer functional purpose from behaviour and structure even with meaningless names. The analysis should continue using structural/behavioural analysis (and dynamic analysis if needed) rather than being abandoned at the sight of renamed identifiers.

26. **Q26.** *You are asked whether a SIM card recovered without its original handset has any independent evidentiary value. Answer with reference to specific data fields.*

Model Answer: Yes — a SIM/UICC card is an independent evidentiary source (Part III). Even without the handset, it can yield the ICCID (identifying the card itself), the IMSI (identifying the subscriber for correlation with carrier records), LOCI/EF-LOCI data (last registered network area), the FPLMN list (networks that refused it service), and potentially SMS remnants directly stored on the card's own limited storage, independent of whatever handset it was most recently used in.

27. **Q27.** *A malware sample's manifest requests no dangerous permissions at all, yet dynamic analysis on an isolated network shows it exfiltrating device data. How is this possible?*

Model Answer: The app likely uses a covert/indirect channel that does not require a declared dangerous permission — for example, abusing a permission it does hold in a way not obviously tied to exfiltration (e.g. INTERNET, which is a normal-level permission, combined with data collected via other legitimately-granted normal permissions), or loading additional executable code at runtime (DexClassLoader) whose true permission requirements were not apparent from the static manifest alone. This reinforces why dynamic analysis remains necessary even when static permission review looks clean.

28. **Q28.** *A colleague argues that Safe Exam Browser lockdown alone is sufficient to prevent AI-assisted cheating in an online exam, without needing carefully designed questions. Do you agree?*

Model Answer: Not entirely. Lockdown software (Part on exam administration) closes the most obvious channel (switching tabs/apps on the same device), but does not prevent a second physical device from being used, nor does it change the fact that a generic, definition-level question is answerable by an AI system from general training knowledge regardless of lockdown. Carefully designed, artefact-specific and scenario-based questions (of the kind in this guide) reduce the value of any successful lookup, complementing rather than being replaced by lockdown software — both layers matter together.

29. **Q29.** *In the capstone-style investigation, the clean and suspect APKs are found to have IDENTICAL smali output for the login flow, but different runtime network behaviour. What does this tell you about where the malicious modification most likely occurred?*

Model Answer: This strongly suggests the malicious behaviour was not injected into the app's static bytecode at all, but is instead being introduced at runtime — most plausibly via a packer/loader mechanism (Part IX) that downloads or decrypts additional malicious logic in memory after the app starts, or via a malicious native library or dynamically loaded component not captured in the disassembled classes.dex. This is exactly the scenario where dynamic analysis (Part X), not further static diffing, is the necessary next step.

30. **Q30.** *Explain why a forensic examiner would want to know an app's targetSdkVersion even when the phone's Android OS version is already known.*

Model Answer: Because Android's runtime security behaviour for a given app is governed largely by what that specific APK targets, not solely by the OS version installed on the device — a device running a modern OS can still host an app targeting a much older API level, which may behave with weaker default protections (component export defaults, permission grant model) than the OS itself is capable of enforcing for modern-targeting apps. Knowing the app's own targetSdkVersion is therefore necessary to correctly assess what security behaviour actually applied to that specific app.

31. **Q31.** *A defence argument claims a forensic finding is invalid because the examiner used a 'toolbox' of three different tools rather than one 'gold-standard' tool. How should the examiner respond?*

Model Answer: The examiner should explain that using multiple independent tools/methods to cross-validate a finding (Part II) is standard, recommended professional practice, not a weakness — no single commercial tool is universally authoritative across every device and data type, and relying on only one tool without independent corroboration would itself be the more questionable methodological choice. The toolbox approach strengthens, rather than undermines, the defensibility of the finding.

32. **Q32.** *You discover that an app's SQLite database file has recently been vacuumed (SQLite's VACUUM command, which rebuilds the file and can reclaim/overwrite freelist space). What is the forensic significance?*

Model Answer: A VACUUM operation can overwrite or eliminate residual data sitting in freelist/unallocated pages (Part IV) that might otherwise have contained recoverable fragments of previously deleted records, meaning any deleted-record recovery attempt against this database is now far less likely to succeed. Discovering that a VACUUM occurred, and roughly when, is itself a potentially significant finding — especially if its timing correlates suspiciously with other case events, since it could represent either routine housekeeping or a deliberate evidence-reduction action worth flagging as a possible anti-forensic indicator (Part IV).

33. **Q33.** *A student is asked to name the single biggest practical difference between doing static analysis on a sample from Lab 5's own build vs. an unknown malware sample found in the wild. What is the correct answer?*

Model Answer: Network isolation. Static analysis technique (unzip, baksmali, manifest review) is identical either way, but an unknown malware sample must never be run or even opened on a machine with live internet access, and any dynamic follow-up must occur strictly on an isolated, host-only network segment (Part XII) — a precaution that is unnecessary for a student's own known, benign Lab 5 build.

34. **Q34.** *Explain, in one sentence each, why 'seizure,' 'acquisition tier,' and 'reporting' each independently determine whether a piece of mobile evidence will hold up in court.*

Model Answer: Seizure determines whether the evidence was lawfully and defensibly obtained in the first place, regardless of what is later found on the device. Acquisition tier determines what categories of data could even theoretically have been recovered, setting the honest boundary of what any finding can claim. Reporting determines whether the findings are communicated with accurate, appropriately-qualified certainty to a non-technical decision-maker (court, panel, client) without overstating what the evidence actually supports.

35. **Q35.** *A Drozer session shows a target package has zero exported components at all. What does this mean for the assessment, and what should the tester do next?*

Model Answer: It means the app presents no directly reachable inter-app attack surface via Android's standard component-export mechanism, which is a genuinely strong defensive posture for that specific vector — but it does NOT mean the app is free of vulnerabilities overall. The tester should shift focus to the remaining attack surfaces covered elsewhere in this guide: server-side/API testing (Part XXV), data-storage review (Part IX/XXIII shared_prefs/SQLite), and dynamic/runtime analysis (Part X) for issues not tied to exported components at all, such as insecure local storage or weak transport security.

36. **Q36.** *You discover an app derives its database encryption key by hashing the device's IMEI. Explain two distinct problems with this approach.*

Model Answer: First, the IMEI is not a secret — it is readable by many apps with appropriate permissions and, on some Android versions/OEMs, via other accessible means, so deriving a 'secret' key from a non-secret value provides no real confidentiality once an attacker can also read the IMEI. Second, the IMEI is fixed for the life of the device, meaning the derived key can never be rotated or changed without changing the underlying device itself, so a single key compromise (or a single flaw in the derivation logic being discovered) compromises the scheme permanently for that device with no remediation path short of re-architecting the whole encryption approach.

37. **Q37.** *A defence attorney claims that because a chip-off extraction is 'destructive,' it must be forensically unreliable. How should you respond?*

Model Answer: Destructive and unreliable are not the same thing: chip-off's destructiveness refers to the device's physical/functional state afterward (it generally cannot be reassembled into a working phone), not to the accuracy or forensic soundness of the data read from the extracted chip. Properly performed chip-off acquisition, with correctly documented handling and verified against expected flash-controller/ECC (error-correction) behaviour for that chip type, is an established and accepted technique precisely because it is used only when non-destructive methods (Part II, XV, XXVIII) have failed or are unavailable, not because it is inherently less accurate.

38. **Q38.** *During a live incident, the response team observes suspicious traffic from a phone for eight minutes on an isolated tap before isolating the device. A stakeholder later asks why the team didn't isolate immediately. What is the strongest justification you could offer, assuming the observation window was pre-approved and tightly scoped?*

Model Answer: The strongest justification is that the brief, controlled, isolated-network observation window was a deliberate, pre-approved trade-off (Part XXIX) made to capture the exfiltration destination and scope of compromise — information that materially improved the team's ability to assess the full extent of the incident and support any subsequent legal/law-enforcement action — while the observation itself occurred on a tap the team controlled, not the live production network, so no additional real-world harm resulted from the brief delay in isolating the device itself.

39. **Q39.** *A junior analyst writes an exam answer that correctly identifies 'file-system acquisition' as the right tier for a given case, but never explains why. Under a typical scenario-based marking rubric, what mark outcome would you expect, and why?*

Model Answer: Partial credit at best. Under rubric-aligned answering (Part XXX), naming the correct technique is necessary but not sufficient — the justification tying the specific case facts to why that technique (and not a shallower or deeper tier) is appropriate is usually where the majority of the available marks sit, since it demonstrates actual reasoning rather than recall of a term. An answer with the right label and no justification typically scores similarly to, or only marginally better than, a wrong label with strong reasoning, depending on the specific rubric weighting.

40. **Q40.** *You are asked whether Drozer alone is sufficient for a complete Android app security assessment. Answer with reference to at least two attack surfaces Drozer does not directly address.*

Model Answer: No — Drozer is focused on the exported inter-app component attack surface (Activities, Services, Receivers, Providers) and is not designed to directly assess: (1) server-side/backend API vulnerabilities (Part XXV), which require proxy-based traffic analysis rather than on-device component testing; and (2) local data-storage security (Part IX/XXIII) such as unencrypted shared_prefs or poorly managed cryptographic keys, which require direct file-system/code review rather than component invocation. A complete assessment combines Drozer with proxy-based dynamic analysis, static code/manifest review, and data-storage inspection.

41. **Q41.** *A case requires evidence from a feature phone so old that no commercial forensic tool lists it as supported. What general strategy, consistent with the Iceberg Theory and toolbox principles covered earlier, should guide your approach?*

Model Answer: Treat the absence of official tool support as a reason to fall back on more manual, foundational techniques rather than as a reason to conclude nothing is recoverable: consult vendor-specific documentation and cable/protocol references for that specific model (Part XXI), consider whether a hardware-level technique (JTAG, or chip-off as a last resort, Part XXVIII) is viable given the device's age and construction, and apply the Iceberg Theory mindset (Part III) that surface/default methods will likely reveal little, meaning deeper, more manual analysis is where the real recoverable data is most likely to be found on unsupported legacy hardware.

42. **Q42.** *An app's Content Provider is protected by a permission that the app itself also requests and is auto-granted to itself at install (a common misconfiguration). What is the practical security consequence of this pattern?*

Model Answer: If the protecting permission is defined and simultaneously requested by the same app without appropriate protection level restrictions, other apps may be able to also request and be granted that same permission (depending on its declared protectionLevel), effectively defeating the protection entirely — any app willing to declare a request for that permission gains the same access the Content Provider's author intended to restrict. The correct pattern is a signature-level permission defined by the protecting app, ensuring only apps signed with the same certificate can ever hold it, rather than a normal or dangerous-level permission that any app can freely request.

43. **Q43.** *In a report, you are asked to state a confidence level for a finding based on a single, uncorroborated artefact. What is the professionally appropriate way to phrase this, drawing on the ethical standards discussed earlier?*

Model Answer: The finding should be stated with explicitly qualified confidence, naming both what was found and that it rests on a single source — for example, 'a single artefact indicates X; this has not yet been independently corroborated by a second source, and should be treated as an initial indication rather than a confirmed finding' — rather than presenting it with the same confidence level as a fully corroborated, multi-source finding. This directly reflects the ethical obligation (Part XIV) not to overstate certainty beyond what the evidence actually supports.

44. **Q44.** *A student argues that since both JTAG and file-system acquisition can sometimes recover similar categories of data, the choice between them doesn't matter. Correct this reasoning.*

Model Answer: The choice matters even when the recoverable data categories overlap, because the two techniques sit at very different points on the invasiveness/risk spectrum and have different prerequisites: file-system acquisition is software-based and non-destructive, typically requiring some degree of device access/trust already established, while JTAG is a hardware-level technique used specifically when software-based methods are unavailable, and carries its own equipment/access requirements and risk profile. Choosing the least invasive method that is actually viable for

the specific device's state (locked/damaged/software-inaccessible) remains the governing principle (Part XV, XXVIII), regardless of any overlap in what data each COULD theoretically recover.

45. **Q45.** *Why does this guide repeatedly pair a 'what happened' technical question with a 'how do you explain this to a non-expert' follow-up across multiple units?*

Model Answer: Because the module's exam is explicitly scenario- and application-based, and real mobile security/forensics work virtually always requires translating a technical finding for a non-technical decision-maker (a court, an executive board, a client) as part of the same task — testing only the technical recall would not reflect how the skill is actually used, and a technically correct finding that cannot be clearly communicated has limited real-world value in this field.

46. **Q46.** *A newly hired analyst insists on using only the single 'best' commercial tool available for every case, rejecting the toolbox approach as 'inefficient.' Using concepts from at least two different parts of this guide, explain why this position is risky.*

Model Answer: This position conflicts with toolbox-forensics practice (Part II, XV), which exists specifically because no single tool is authoritative across every device/data type, and skipping cross-validation risks an unchecked parsing error becoming a reported 'fact.' It also conflicts with the professional/ethical standards discussed in Part XIV, since presenting a single-tool finding with full confidence, without independent verification, risks the same overstatement-of-certainty failure discussed there — efficiency is a legitimate operational concern, but it does not override the methodological requirement to corroborate significant findings before they are relied upon.

47. **Q47.** *Summarise, in a single sentence, the common thread linking Parts XXVI (Drozer workflow), XXX (exam technique), and II (toolbox forensics).*

Model Answer: All three emphasise that a correct, defensible outcome depends on following a disciplined, ordered METHODOLOGY (recon before exploitation; structure before content recall; cross-validation before conclusion) rather than on any single tool, fact, or technically correct-sounding statement in isolation.

PART XXXIV: ESSAY-STYLE PRACTICE QUESTIONS

Full-length, multi-part scenario questions simulating the long-answer format of a scenario/application-based exam. Practise writing a complete answer (≈15–20 minutes each) before reading the model answer — note how each model answer moves through seizure/setup, method choice with justification, execution, corroboration, and conclusion, in that order.

Essay Question 1

A mid-sized company suspects an employee used a company-issued Android phone to leak confidential product designs to a competitor via a messaging app, then attempted to delete the evidence. You are the lead examiner. Describe your full investigative approach from seizure through to your final report, explicitly naming which acquisition tier(s) you would use and why, how you would attempt to recover the deleted messages, how you would corroborate your findings, and how you would structure your final report for a non-technical executive board.

Model Answer

Seizure: upon authorisation to examine the company-owned device, I would isolate it from all networks immediately (Airplane Mode or Faraday enclosure) to prevent remote wipe or further deletion, photograph its current state, and complete a chain-of-custody form treating the handset as the primary evidence item. I would document whether the device is in an AFU or BFU state, since this determines the acquisition options realistically available. Acquisition: given the requirement to recover DELETED messages, a purely logical acquisition (adb backup/pull) would be insufficient, since logical extraction only reaches currently-accessible data through OS APIs. I would attempt a file-system-tier acquisition first, since it can recover residual SQLite WAL/journal fragments and cached data without the full risk/complexity of a physical acquisition; if the case's evidentiary requirements and available time/legal authority justify it, and file-system acquisition proves insufficient, I would escalate to physical acquisition, understanding this carries the highest technical risk and difficulty on a modern encrypted device. Recovering deleted messages: I would locate the messaging app's SQLite database and specifically check for an accompanying -wal or -journal file, since recently deleted or modified records can persist there even after removal from the main committed database; I would also check for unvacuumed freelist/unallocated pages within the main database file itself for residual fragments of deleted rows. Corroboration: rather than relying on the recovered message content alone, I would cross-reference timestamps against system logs, shared_prefs 'last used' flags for the messaging app, and (if available) the company's own network/DLP logs showing outbound data transfer around the same timestamps, since independent corroboration across multiple artefact types is far more defensible than any single recovered message. I would also independently validate any commercial tool's parsed output by manually querying the SQLite database directly, consistent with toolbox-forensics practice. Reporting: my final report would open with a plain-language executive summary stating what was found and its significance without technical jargon, followed by a methodology section documenting every tool, command, and hash value used (for independent reproducibility), a findings section presenting the corroborated timeline, and a clearly stated limitations section acknowledging what the chosen acquisition tier could and could not recover — with the full technical detail (raw SQL queries, hash manifests, command logs) placed in appendices rather than the main narrative, so the executive board can act on a clear conclusion while a technical reviewer can independently verify every step.

Essay Question 2

You are conducting a security assessment of a banking app. Static analysis reveals the app is protected by ProGuard and a commercial packer. Explain how you would proceed with your assessment, why your usual static-analysis workflow will be insufficient, and how you would specifically test for (a) certificate pinning bypass and (b) exported-component vulnerabilities, tying each back to the relevant OWASP Mobile Top 10 categories.

Model Answer

Because the app is both ProGuarded and packed, my usual static workflow (unzip → baksmali → read smali) will yield renamed, hard-to-follow identifiers at best, and at worst a largely meaningless loader stub if the real payload is encrypted and only unpacked at runtime — attempting to push through with more static analysis against a packed sample is a known trap, since packing is specifically designed to defeat exactly that workflow. I would therefore shift

the bulk of my effort to dynamic analysis: running the app in an instrumented emulator or rooted test device with Frida attached, allowing me to hook methods and inspect the real, decrypted logic in memory once the packer's runtime unpacking routine has executed, rather than trying to recover it from the static classes.dex. For (a) certificate pinning: I would first attempt standard proxy interception (Burp Suite/mitmproxy with the CA certificate trusted on the test device); if no traffic appears despite a correctly trusted certificate, this is the standard signature of pinning, and I would use a Frida script (or a tool such as AndroidSSLTrustKiller) to hook and neutralise the app's pinning-check method at runtime before re-attempting interception. A successful bypass revealing sensitive data in transit maps to OWASP M3 (Insufficient Transport Layer Protection), and if the token itself lacks proper expiry/rotation once observed, secondarily to M9 (Improper Session Handling). For (b) exported components: I would run Drozer's `app.package.attacksurface` module to enumerate all exported Activities, Services, Receivers, and Content Providers, then systematically probe each with the relevant Drozer module (e.g. `app.provider.query` with a malformed selection string to test for SQL injection, per the standard recon → probe → exploit pattern). A successful Content Provider injection maps most directly to M7 (Client-Side Injection), while an exported component reachable via untrusted Intent data more broadly maps to M8 (Security Decisions via Untrusted Inputs). Throughout, I would document each finding's OWASP mapping explicitly in my report, since this is the standard framework reviewers and clients expect for prioritising remediation.

Essay Question 3

Compare and contrast the forensic acquisition of a modern Android device versus a modern iPhone, in the context of an investigation where the device's lock/unlock state at seizure is unknown until examined. Discuss what determines the achievable acquisition tier on each platform, and explain why the AFU/BFU concept, while named in relation to iOS, is conceptually relevant to Android as well.

Model Answer

On Android, the achievable acquisition tier is shaped primarily by whether USB debugging is already enabled and authorised (a prior trust relationship with a specific computer, similar in spirit to iOS's trusted pairing), whether the device is unlocked or a passcode/biometric is available, and the specific OEM/model's support in available forensic tools given Android's device fragmentation. A device with USB debugging already enabled and unlocked (or with a known unlock method) can often support at least a logical, and frequently a file-system, acquisition; a locked device with debugging disabled and no known passcode is far more restrictive, potentially limiting acquisition to whatever a locked-state extraction method the specific OEM/tool combination supports. On iOS, Apple's tightly closed ecosystem and default encryption make the AFU/BFU distinction the single most decisive factor: an AFU device (unlocked at least once since boot) combined with an existing trusted-computer pairing or known backup password can permit substantially deeper extraction via established iOS acquisition methods, while a BFU device with neither is typically limited to whatever data-class-based encryption exposes before first unlock — often very little. Although the AFU/BFU terminology is drawn from iOS's specific encryption lifecycle, the underlying concept transfers directly to Android: an Android device that has never been unlocked since boot, or has lost an existing trust relationship (e.g. via a factory data reset or revoked ADB authorisation), sits in an analogous 'locked-since-boot' state with substantially reduced acquisition options compared to one already unlocked with an established trust relationship. In both cases, the device's unlock/trust state at the moment of seizure is a more decisive variable for planning acquisition than the specific brand of tool used, which is why documenting that state accurately during seizure (Part I, XVIII) is critical to setting realistic expectations for what the acquisition phase can actually achieve.

Essay Question 4

You are given a suspect APK and asked to determine whether it is a trojanised version of a known-good app. Describe your complete methodology, explicitly integrating static comparison, dynamic observation, OWASP Top 10 classification of any finding, and how you would present your conclusion to a non-technical client who only wants to know 'is this app safe to keep using.'

Model Answer

Methodology: I would begin by obtaining a verified-clean reference copy of the known-good app (from the official store or a trusted archive) and unzip both it and the suspect APK to compare their AndroidManifest.xml files first, looking for any added permissions, exported components, or altered package/signing metadata — a fast, high-value

first pass. I would then disassemble both classes.dex files with baksmali and diff the resulting smali directories, focusing specifically on the login/authentication and any data-handling code paths, since malicious repackaging most commonly targets exactly these flows (as in the Part VIII worked example). Any structural difference found (an added Log.d call, an added network call, an added permission) becomes my primary static finding. I would then move to dynamic analysis: running the suspect APK in an isolated AVD/rooted device with a proxy and Frida attached, to observe actual runtime behaviour and confirm whether the statically-identified difference actually executes and produces the suspected effect (e.g. credentials genuinely appearing in Logcat, or a genuine unexpected outbound connection). I would classify any confirmed finding against the OWASP Mobile Top 10 (e.g. M4 Unintended Data Leakage for a logging leak, M2 Insecure Data Storage if credentials are also cached insecurely) to frame its severity in an industry-standard way for any technical audience. For the non-technical client, I would translate the finding into a single plain-language verdict — for example, 'this version of the app has been modified to secretly record your login details; it is not safe to keep using, and we recommend uninstalling it and changing any passwords entered while it was installed' — deliberately omitting OWASP codes, smali diffs, and tool names from that headline answer, while making the full technical detail available in an appendix for anyone who wants to verify the finding independently.

Essay Question 5

A regulator asks your organisation to justify why a mobile forensic examiner's testimony should be trusted over a defence expert's competing interpretation of the same extraction. Construct the strongest possible response, drawing on the professional/ethical standards, toolbox-validation practice, and reporting principles covered in this guide.

Model Answer

The strongest response rests on process, not personal authority: the examiner's findings should be trusted to the extent that the process behind them is transparent, repeatable, and independently checkable — not because of the examiner's title. Specifically: (1) the examiner operated under a recognised professional code (IACIS/ISFCE-aligned) governing objectivity and disclosure of limitations, rather than asserting unqualified certainty; (2) any significant finding was cross-validated using the toolbox approach — a second independent tool or manual method corroborating the primary tool's output — rather than resting on a single vendor's parser; (3) the acquisition tier used, and what it could and could not recover, was documented and disclosed rather than left implicit, so the finding's boundaries are known rather than assumed; and (4) the full methodology (tool versions, exact commands, hash values) was recorded in enough detail that a genuinely independent examiner — including, in principle, the defence's own expert — could reproduce the same steps against the same original evidence and arrive at the same result. A competing interpretation should be evaluated on whether it can be reproduced through this same transparent process, not on which expert is more senior or persuasive; if the defence expert's competing interpretation cannot be reproduced from the documented methodology and evidence, that is the substantive basis for preferring the original finding — not appeal to authority.

Essay Question 6

Design a triage checklist a junior analyst could follow, without supervision, to make an initial (non-final) call on whether a newly submitted APK sample warrants escalation to full malware analysis. Justify each checklist item by reference to the indicator patterns and traps covered in this guide.

Model Answer

1) Read AndroidManifest.xml and list all requested permissions; flag any permission with no obvious connection to the app's stated category (Part XII/XXIV — permission/purpose mismatch is the single fastest high-value signal). 2) Check exported components and their protection levels (Part VI/XI/XVII); flag any exported component with no permission requirement, since this defines the app's real external attack surface regardless of intent. 3) Disassemble classes.dex with baksmali and search for DexClassLoader or other reflective/dynamic code-loading calls, and for hard-coded IP addresses/URLs (Part XII); flag any hit, but do NOT treat a single hit alone as conclusive (Part XXIV — composite pattern reasoning, avoiding false positives from legitimately obfuscated/reflective apps). 4) Note the obfuscation level relative to the app's apparent complexity; heavy obfuscation on a trivially simple-purpose app is a mild additional signal, but only when combined with at least one other item above. 5) Escalate to full analysis (which must occur on an isolated, host-only network segment — Part XII, non-negotiable) if at least TWO independent items

from steps 1–4 are flagged together; a single flagged item alone should be logged for awareness but not treated as sufficient grounds for an urgent escalation, consistent with the false-positive risk discussed in Part XXIV.

Essay Question 7

A university is designing its own scenario-based practical exam for a mobile security and forensics course, modelled on the structure of this guide. Drawing on everything covered here, propose a single exam scenario (you may invent plausible case details) that would require a student to draw on BOTH the forensics track and the hacking/RE track within one connected answer, and explain what a full-marks response would need to demonstrate at each stage.

Model Answer

Proposed scenario: 'A university IT department suspects a student's personal Android phone, connected to the campus Wi-Fi, was used to run a modified version of the official student-portal app that silently captured other students' login credentials entered on the same shared kiosk device before the phone was later confiscated under the university's acceptable-use policy. Investigate.' A full-marks response would need to demonstrate: (1) correct legal/procedural framing under an institutional (rather than criminal/warrant) authority — acceptable-use policy scope rather than a criminal warrant, correctly distinguishing the two; (2) appropriate seizure/isolation steps for a device that may still be actively capturing data; (3) a justified acquisition-tier choice given the case's specific evidentiary needs (likely file-system, to recover both the modified APK and any locally cached captured credentials); (4) comparative static analysis between the official portal app and the version found on the phone, correctly identifying the credential-capture modification using the disassembly workflow; (5) correct OWASP Top 10 classification of the finding (most likely M4 Unintended Data Leakage, or M7 if the capture mechanism itself involved a client-side injection vector); (6) dynamic confirmation of the capture behaviour in an isolated environment before concluding it is real rather than a purely static/theoretical possibility; and (7) a properly structured report distinguishing the plain-language institutional finding (what happened, who was affected, recommended remediation for affected students) from the full technical appendix. A response earning full marks would need to move through all seven elements in a coherent, correctly-ordered narrative, not merely list them — exactly the exam technique described in Part XXX.

Essay Question 8

Reflecting on the entire guide, identify the THREE concepts you believe are most likely to appear, in some form, across multiple different question types (MCQ, one-word, scenario, and essay) in your actual exam, and justify your selection using evidence from how frequently each concept recurs across this guide's units, scenario bank, and essay bank.

Model Answer

This is a self-reflection exercise with no single correct answer, but a well-reasoned response should identify concepts that demonstrably recur across MULTIPLE guide sections, not just multiple mentions within one unit. Strong candidates, with justification: (1) The acquisition-tier trade-off (logical/file-system/physical) — it anchors Part II and XV directly, resurfaces in the SIM/legacy-device units (III, XXI), the iOS comparison (XVIII), the advanced physical techniques unit (XXVIII), several scenario-bank questions, and two of the essay questions, making it the single most cross-referenced technical concept in the guide. (2) The 'absence of evidence in a tool's output is not evidence of absence' principle (Trap 2 in Part XX) — it appears explicitly in the toolbox-forensics unit (II), the SQLite WAL/journal unit (IV), the reporting/testimony unit (V/XIV), and is directly named as the guide's single most-repeated meta-lesson at the close of Part XX, plus it reappears in several scenario-bank items about tools reporting 'no data found.' (3) The static-analysis-defeated-by-packing-therefore-switch-to-dynamic-analysis pairing — it structures the relationship between Parts VIII/IX and X directly, is tested explicitly in the scenario bank and in Essay Question 2, and recurs again in the malware-triage unit (XII) and the capstone case study (XIX). A student who can correctly explain and apply these three concepts across different question formats (not just recall their definitions) would be well-positioned for the large majority of this guide's cross-referenced content.

Essay Question 9

You are handed a device that presents BOTH forensic and security-testing questions simultaneously: a company wants to know (a) what data a suspect employee's device contains relevant to a policy violation, AND (b) whether the company's

own internal app, installed on that device, has a vulnerability that let the employee access data they shouldn't have. Explain how you would sequence and separate these two workstreams so that neither compromises the other, referencing the relevant parts of this guide for each.

Model Answer

These two workstreams should be deliberately sequenced and kept methodologically distinct, even though they use the same physical device as a starting point. First, I would complete the forensic evidentiary workstream in full: proper seizure and documentation (Part I, XIV), a justified acquisition-tier choice (Part II, XV) preserving the device/extraction as unaltered evidence, and analysis/reporting (Part IV, V) addressing question (a) — what data exists and what it shows about the policy violation — treating this as the evidentiary record that must remain defensible and untouched by any subsequent security testing. Only once that evidentiary extraction is complete and preserved (hashed, documented, set aside) would I begin the SEPARATE security-assessment workstream for question (b): obtaining a fresh, disposable test copy of the company's internal app (not the seized device's live installation) and testing it in an isolated lab environment using the static/dynamic techniques from Parts VI–XIII, XVI–XVII, XXIII, and XXV, specifically to determine whether an exported-component flaw, an IDOR, or a data-storage weakness could explain how the employee accessed data beyond their authorisation. Keeping these workstreams separate matters because actively exploiting/testing against the SAME physical seized device risks altering or contaminating the evidentiary record relied upon for question (a), and because the two questions have different audiences and different reporting formats (Part V/XIV's evidentiary report structure for (a); a standard vulnerability/remediation report, likely referencing the OWASP Top 10 per Part XIII, for (b)).

PART XXXV: GLOSSARY OF CRITICAL TERMS

An alphabetised consolidation of every critical term across both tracks. If you cannot define a term here in one sentence from memory, return to its source unit.

Term	Definition
Acquisition tier	The depth/method of extraction: logical, file-system, or physical, each with different recovery capability and risk.
ADB	Android Debug Bridge — the primary host-to-device command bridge for Android.
AFU / BFU	After First Unlock / Before First Unlock — device encryption-accessibility states that gate what can be extracted.
APK	Android application package — a ZIP archive containing compiled code, resources, and manifest.
ART	Android Runtime — the modern execution environment that replaced Dalvik's interpreter/JIT model.
baksmali / smali	Tools (and the intermediate language) for disassembling classes.dex to editable text and reassembling it.
Broadcast Receiver	An Android component that listens for and responds to system- or app-wide broadcast events.
C2 (command-and-control)	A remote server malware contacts for instructions or data exfiltration.
Chain of custody	Documented, unbroken record of evidence handling from seizure to presentation.
Content Provider	An Android component exposing structured data to other apps via a defined query interface.
Custom recovery	A replacement recovery-partition image enabling flashing of arbitrary packages, incl. root.
CVE-2013-6271	A documented Android vulnerability enabling lock-screen bypass without hash-cracking.
Drozer	An Android security-assessment framework for enumerating and exploiting exported components.
Exported component	An Activity/Service/Receiver/Provider reachable by other apps, by explicit flag or implicit intent-filter.
Faraday enclosure/bag	A signal-shielding container blocking cellular/Wi-Fi/Bluetooth to a seized device.
FPLMN	Forbidden PLMN list on a SIM — networks that refused the card service.
Frida	A cross-platform dynamic instrumentation toolkit for runtime hooking.
Freelist page	Unallocated SQLite page that may retain remnants of deleted records.
Heimdall / Odin	Cross-platform / Windows-only flashing tools for Samsung download-mode firmware flashing.
IACIS / ISFCE	Professional bodies for computer/mobile forensic examiner certification and standards.

Term	Definition
Iceberg Theory	The principle that surface extraction reveals only a fraction of a device's recoverable data.
ICCID	Integrated Circuit Card Identifier — uniquely identifies the physical SIM card.
IMSI	International Mobile Subscriber Identity — identifies the subscriber account on the network.
jarsigner / keytool	JDK tools for signing an APK / generating the signing key used to do so.
LOCI / EF-LOCI	SIM location-information file recording the last registered network cell/area.
Logical acquisition	Extraction via OS-exposed APIs; fast and safe but cannot recover deleted data.
META-INF/	APK directory holding the signing certificate; must be removed before re-signing a repackaged APK.
OWASP Mobile Top 10 (2014)	A ranked list (M1–M10) of common mobile app security risk categories.
Physical acquisition	Bit-level storage copy; can recover deleted/unallocated data but is hardest and riskiest to obtain.
ProGuard	Android SDK's built-in code shrinker/obfuscator.
QARK	Quick Android Review Kit — static analysis tool for Android app security issues.
Rollback journal	An alternate SQLite journaling file that can retain residual transactional data.
Shared_prefs	Android's lightweight XML key-value storage mechanism for small app state.
SSL/certificate pinning	An app-level check rejecting any certificate but a specifically expected one, defeating generic proxy interception.
Toolbox forensics	The practice of cross-validating forensic findings across multiple independent tools/methods.
UID sandboxing	Android's per-app Linux user-ID isolation model for file-system access.
Validation vs. verification	Case-specific correctness check vs. general documented-capability check of a forensic tool.
WAL (Write-Ahead Log)	A SQLite journal file holding recent writes not yet merged into the main database file.
zipalign	Tool aligning uncompressed APK data to a byte boundary, run after signing, to reduce runtime RAM use.
Android Keystore system	Android's platform-provided secure storage facility for cryptographic keys, isolated from normal app-accessible storage.
BES (BlackBerry Enterprise Server)	Enterprise management infrastructure for BlackBerry devices; sometimes a necessary acquisition avenue alongside the device itself.
Chip-off acquisition	Physically desoldering and directly reading a device's flash memory chip; destructive and irreversible, a last-resort technique.
Cloud-extraction module	A vendor-specific tool component for retrieving account-level cloud-synced data independent of physical device access.

Term	Definition
Composite indicator pattern	A combination of multiple individually-weak malware signals that together form strong evidence of malicious intent.
Contain vs. observe trade-off	The live-incident-response decision between immediately isolating a compromised device versus briefly observing it under control.
Distractor	An exam answer option or framing designed to be plausible but incorrect, usually by conflating two related concepts.
Drozer agent	The component installed on a target Android device/emulator that the Drozer console communicates with to run assessment modules.
Excessive data exposure	An API returning more data in its response than the client actually needs, risking leakage of sensitive fields.
False positive	An incorrect malicious classification triggered by an innocent app that happens to share one suspicious-looking trait.
IDOR	Insecure Direct Object Reference — a server-side flaw where manipulating an identifier grants access to another user's data.
JTAG	A hardware debug interface standard; used forensically to read raw device flash memory directly without booting the OS.
Multi-device correlation	Cross-referencing evidence appearing on more than one device/account instance to strengthen or challenge a finding.
Predictable key derivation	Generating an encryption key from a fixed, guessable value rather than a securely random or properly derived secret.
Rubric-aligned answering	Structuring an exam answer to explicitly surface the specific, nameable elements a marking scheme is likely to reward.
Vendor-specific cable protocol	A proprietary, non-standard data-transfer protocol required for certain feature-phone extractions, unlike a universal method like ADB.
Android Backup Extractor	A utility that converts a .ab archive into a standard tar file for normal examination.
Overstatement of findings	Presenting a conclusion with more certainty than the underlying method/evidence actually supports — an ethical, not just technical, failure.
Scope of authority	The legally defined boundary (warrant, consent, exigent circumstance) within which a search/seizure is lawful.
Device-support matrix	A vendor's documented list of which devices/OS versions a given acquisition method supports; varies by tool and changes frequently.

PART XXXVI: RAPID REVISION CHECKLIST

Your final-day, no-new-reading pass. Read this list once in the morning of your exam.

1. Four-stage forensic process, in order: Seizure → Collection → Analysis/Examination → Presentation.
2. Three acquisition tiers, in order of depth/risk: Logical < File-System < Physical.
3. ICCID = the card. IMSI = the subscriber. Do not swap them under exam pressure.
4. WAL/journal files can hold data the main .db file does not show — always check for them.
5. Toolbox forensics: verify (general capability) vs validate (this case's correctness) — always cross-check an implausible result with a second method.
6. AFU vs BFU: powering off an unlocked device can push it into a harder-to-access encryption state.
7. Correct RE order: unzip → baksmali → edit smali → smali → delete META-INF → re-zip → sign → zipalign (last) → install.
8. android.util.Log.d is STATIC → invoke-static in smali, never invoke-virtual.
9. ProGuard hides identifiers, NOT string literals/resource names — don't overstate what it protects.
10. Packing defeats STATIC analysis → the correct countermeasure is DYNAMIC analysis, not more baksmali.
11. Certificate/SSL pinning defeats a plain proxy — must be bypassed at runtime before traffic becomes visible.
12. Exported component + no permission = reachable by any app on the device, including a malicious one.
13. Odin = Windows-only; Heimdall = cross-platform. Both are Samsung download-mode flashing tools.
14. gesture.key = SHA1 hash of the pattern lock, recoverable via adb pull + offline cracking.
15. Malware red flag pattern: permission/purpose mismatch + reflective code loading (DexClassLoader) + suspicious runtime network destination.
16. Never run a malware sample with live internet access — isolated/host-only network only.
17. A repackaged/re-signed APK cannot install over the original unless the original is uninstalled first (signature mismatch).
18. Reports: executive summary in plain language FIRST; technical detail goes into appendices.
19. Corroboration across independent artefacts (DB + log + shared_prefs) beats any single artefact alone — always name a second source in a 'how do you confirm this' answer.
20. A rooted device under examination reduces confidence in on-device timestamps/logs — flag this explicitly if relevant.